computers

MDPI

Article

Hybrid LLM-Assisted Fault Diagnosis Framework for 5G/6G Networks Using Real-World Logs

Aymen D. Salman [1], Akram T. Zeyad [1], Shereen S. Jumaa [1], Safanah M. Raafat [2,3], Fanan Hikmat Jasim [1] and Amjad J. Humaidi [2,*]

[1] College of Computer Engineering, University of Technology-Iraq, Baghdad 10066, Iraq; aymen.d.salman@uotechnology.edu.iq (A.D.S.); akram.t.zeyad@uotechnology.edu.iq (A.T.Z.); shireen.s.jumaa@uotechnology.edu.iq (S.S.J.); fanan.h.jasim@uotechnology.edu.iq (F.H.J.)
[2] College of Artificial Intelligence Engineering, University of Technology-Iraq, Baghdad 10066, Iraq; safanah.m.raafat@uotechnology.edu.iq
[3] Cyber Security Techniques Engineering Department, Al Kut University College, Wasit 46137, Iraq
* Correspondence: amjad.j.humaidi@uotechnology.edu.iq

Abstract

This paper presents Hy-LIFT (Hybrid LLM-Integrated Fault Diagnosis Toolkit), a multi-stage framework for interpretable and data-efficient fault diagnosis in 5G/6G networks that integrates a high-precision interpretable rule-based engine (IRBE) for known patterns, a semi-supervised classifier (SSC) that leverages scarce labels and abundant unlabeled logs via consistency regularization and pseudo-labeling, and an LLM Augmentation Engine (LAE) that generates operator-ready, context-aware explanations and zero-shot hypotheses for novel faults. Evaluations on a five-class, imbalanced Dataset-A and a simulated production setting with noise and label scarcity show that Hy-LIFT consistently attains higher macro-F1 than rule-only and standalone ML baselines while maintaining strong per-class precision/recall (≈0.85–0.93), including minority classes, indicating robust generalization under class imbalance. IRBE supplies auditable, high-confidence seeds; SSC expands coverage beyond explicit rules without sacrificing precision; and LAE improves operational interpretability and surfaces potential "unknown/novel" faults without altering classifier labels. The paper's contributions are as follows: (i) a reproducible, interpretable baseline that doubles as a high-quality pseudo-label source; (ii) a principled semi-supervised learning objective tailored to network logs; (iii) an LLM-driven explanation layer with zero-shot capability; and (iv) an open, end-to-end toolkit with scripts to regenerate all figures and tables. Overall, Hy-LIFT narrows the gap between brittle rules and opaque black-box models by combining accuracy, data efficiency, and auditability, offering a practical path toward trustworthy AIOps in next-generation mobile networks.

Keywords: 5G networks; 6G networks; fault diagnosis; network logs; semi-supervised learning; interpretable AI; large language models; explainable AI; network operations

check for updates

Academic Editor: Amit Kumar Mishra

Received: 12 November 2025
Revised: 7 December 2025
Accepted: 9 December 2025
Published: 12 December 2025

Citation: Salman, A.D.; Zeyad, A.T.; Jumaa, S.S.; Raafat, S.M.; Jasim, F.H.; Humaidi, A.J. Hybrid LLM-Assisted Fault Diagnosis Framework for 5G/6G Networks Using Real-World Logs. *Computers* **2025**, *14*, 551. https://doi.org/10.3390/computers14120551

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The accelerating deployment of 5G and development of 6G networks has heightened the importance of timely and accurate fault diagnosis in telecommunication systems. Modern mobile networks are extraordinarily complex and dynamic systems, incorporating virtualized and heterogeneous infrastructure to support services from enhanced broadband to ultra-reliable low-latency communications [1]. These systems produce vast volumes of log data capturing events, performance metrics, and error messages. Manually inspecting

such logs for faults is infeasible at scale, yet ensuring network reliability is mission-critical. Traditional network Operations and Maintenance (O&M) teams have relied on expert-defined rules and threshold-based alarms to detect faults. However, threshold rules and human expertise alone struggle to keep up with modern networks' complexity [2]. Prior studies note that conventional methods based on static expert knowledge tend to be subjective and inconsistent, failing to meet the needs of contemporary network maintenance [2]. As 5G/6G networks evolve, there is a pressing need for intelligent fault diagnosis tools that can automatically learn from data while remaining interpretable and trustworthy [1].

Machine learning (ML) approaches have shown promise in network fault detection, outperforming purely manual or rule-based systems in several cases [2]. For example, deep learning models trained on large-scale network measurements achieved significantly better detection performance than traditional expert systems [2]. Nevertheless, fully supervised ML requires abundant labeled fault data, which is often scarce or expensive to obtain in networking domains [3]. Telecommunications faults, especially rare or novel failure modes, may not have enough historical examples for training robust supervised models [4]. Recent research has highlighted label scarcity as a major challenge, spurring techniques like data augmentation [4] or digital twins to synthesize labeled data [2]. Furthermore, many high-performing ML models act as black boxes, providing little insight into why a particular alarm was raised. In critical infrastructure like mobile networks, explainability and accountability of AI decisions are paramount [1]. Network operators require interpretable diagnoses to trust and rapidly act on automated fault analysis.

While rule-based O&M systems offer high precision for recurring, well-understood failure patterns, they cannot capture the dynamic and evolving nature of faults in modern 5G/6G networks. New failure modes emerge after software updates, vendor configuration changes, or unexpected interactions between virtualized components, and these patterns often do not satisfy any predefined rule. Machine learning is necessary because it learns statistical representations of log behaviour, enabling the detection of faults that partially manifest, deviate from known signatures, or have never been previously labeled. Prior research in network analytics has repeatedly demonstrated that ML-based models outperform static rules by generalizing beyond rigid patterns and adapting to noisy, heterogeneous log streams. Our use of ML is therefore driven by technical necessity rather than trend; it expands coverage, improves recall under label scarcity, and provides adaptability under concept drift capabilities that rule-based systems alone fundamentally lack. By combining ML with interpretable rules, Hy-LIFT achieves both reliability for known faults and scalability for unforeseen ones.

To address these gaps, we propose Hy-LIFT (Hybrid LLM-Integrated Fault Diagnosis Toolkit), a novel fault diagnosis framework that hybridizes symbolic and machine-learning techniques, augmented by the power of large language models. Hy-LIFT is designed with the following guiding objectives: (i) achieve high accuracy by combining rule-based precision with learning-based recall; (ii) remain interpretable and transparent, so that each detected fault is accompanied by a human-understandable rationale; (iii) cope with limited labels and class imbalance via semi-supervised learning on real log data; and (iv) handle novel or previously unseen faults through zero-shot generalization and explanation using an LLM. In essence, Hy-LIFT seeks to retain the strengths of expert systems (interpretability, precision on known issues) while mitigating their brittleness by incorporating data-driven and language-driven intelligence for the unknown.

Contributions: This work makes the following key contributions:

- **Hybrid Fault Diagnosis Framework:** We design the multi-stage Hy-LIFT pipeline that integrates an interpretable rule-based engine (IRBE) for known fault detection, a semi-supervised classifier (SSC) that learns from both labeled and unlabeled logs,

and an LLM Augmentation Engine (LAE) that provides explanatory and zero-shot diagnostic capabilities. To our knowledge, this is the first framework to tightly couple rule-based reasoning, semi-supervised learning, and LLM-based analysis for network fault management.

- **Interpretability and Explainability:** The IRBE module encodes expert domain knowledge as human-readable rules, ensuring that detections of known faults are immediately explainable by design (each triggered rule corresponds to a condition in the network). The LAE module leverages a large language model to generate natural language explanations for both known and novel faults, translating low-level log patterns into high-level insights. We demonstrate that the combined system produces rich explanations that are understandable to engineers, addressing the trust gap in ML-driven O&M [1].
- **Robust Semi-Supervised Learning:** We develop a semi-supervised fault classifier that uses pseudo-labeling to make use of real-world log data where labels are scarce. The SSC is initially seeded with high-precision labels from IRBE and iteratively improves its coverage by labeling new samples with a confidence-based mechanism [5]. This approach exploits large unlabeled log corpora to enhance the detection of subtle or latent fault patterns that rules alone might miss. We show that our SSC achieves higher recall and F1 than a purely supervised classifier trained on the limited available labels, consistent with other log analysis works that saw F1 improvements over 180% via probabilistic label estimation [6].
- **Evaluation on Real Network Logs:** We evaluate Hy-LIFT on real-world 5G/6G network log datasets (described in Section 4) containing a variety of fault types (e.g., hardware failures, congestion events, handover failures, backhaul outages). The evaluation includes overall and per-class performance metrics (accuracy, precision, recall, F1) under different conditions, comparison against baseline methods (pure rule-based, pure ML), and analysis of robustness to noise and class imbalance. We also include case studies of novel fault incidents to assess the LLM's zero-shot diagnostic capability. All results are presented with a focus on reproducibility and statistical significance.
- **Open-Source Implementation:** To encourage transparency and further research, we provide an outline of our implementation, with all critical components reproducible. (Author's note: repository and data links omitted for review.) The framework is implemented using open-source libraries and can be integrated into existing network management systems. We adhere to principles of scientific rigor, carefully validating each component, and discuss how the approach can be deployed in practice (including considerations of runtime performance and maintaining the system as networks evolve).

The rest of this paper is set up as follows: Section 2 looks at other work in network fault diagnosis, such as rule-based systems, machine learning methods, and new LLM-based analysis methods. Section 3 goes into more detail about the Hy-LIFT framework, including the IRBE, SSC, and LAE parts and how they all work together. Section 4 shows the experimental setup and the results of testing on real-world logs. These results include both quantitative performance and qualitative explanation analysis. Section 5 goes into great detail about the results, the framework's strengths and weaknesses, and practical issues like concept drift and deployment. The paper ends with Section 6, which also talks about possible future research. At the end, there is metadata about the author's contributions, funding, data availability, and conflicts of interest.

2. Related Work

2.1. Traditional and Rule-Based Network Fault Diagnosis

Early methods for managing network faults depended a lot on expert systems based on rules and alarms with fixed thresholds. In traditional cellular networks (e.g., 3G/4G), operators used their knowledge of the field to make if–then rules that could find known failure conditions [2]. For instance, a rule might trigger an alarm if a Key Performance Indicator (KPI) exceeds a threshold (e.g., drop call rate > 2% over a 15 min window). Rule-based systems benefit from interpretable decision logic; their chain of reasoning can be directly understood by engineers [7]. However, as networks grew in scale and complexity, purely manual rule creation became impractical. Static rules often failed to generalize and required constant tuning; studies reported that expert-defined methods suffered from subjectivity and inconsistency across different experts and environments. Barco et al. (2015) applied genetic fuzzy logic to automate rule optimization in LTE self-healing systems [8], and Szilágyi & Nováczki (2012) proposed an expert system for mobile network diagnosis [9], but these still needed significant expert input and were limited to known fault signatures.

Hybrid strategies emerged to augment rule-based systems with data-driven components. Notably, Uszko et al. (2023) introduced a rule-based wireless IDS for 5G WLANs that incorporates machine learning to detect new threats [10]. Their system uses a modular rule engine for known Wi-Fi attacks and trains ML models for anomaly detection of emerging attacks. This combination achieved high accuracy (~98.6%) and recall (92%) in detecting wireless threats, illustrating the benefits of a hybrid approach. Our proposed IRBE module plays a similar role of high-precision detection for known faults, establishing a reliable baseline using domain knowledge. However, unlike security IDS rules (which target malicious events), our IRBE targets network performance and reliability faults (e.g., equipment failures, congestion indicators) in cellular systems. Furthermore, Hy-LIFT extends beyond prior rule + ML systems by integrating an LLM for explanation and novel fault analysis, which has not been addressed in earlier rule-based telecom fault systems.

Another recent trend is knowledge-driven fault diagnosis that fuses expert knowledge with ML. Zhao et al. (2024) developed a knowledge-and-data fusion algorithm for 5G networks that first uses a Naive Bayesian model (augmented with expert rules) to pre-diagnose faults and generate an interpretable association graph, then feeds this graph into a Graph Convolutional Network (GCN) for final classification [4]. This approach achieved ~90.6% accuracy and 88.4% macro-F1 on real 5G cell fault data. Notably, the combined use of a Bayesian expert model and GCN outperformed either alone, demonstrating the value of complementary knowledge-driven and data-driven stages. Hy-LIFT shares a similar philosophy of complementary components: our IRBE (analogous to their Bayesian pre-diagnosis) handles one part of the problem with high confidence, while the SSC (analogous to their GCN) learns from data to improve coverage. A key difference is that Zhao et al. focused on augmenting training data via GANs to address class imbalance, whereas we focus on pseudo-labeling real unlabeled logs and on providing LLM-based explanations for results. Both approaches emphasize interpretability: Zhao et al.'s association graphs provide engineering insights, and our IRBE/LLM components similarly ensure the diagnosis process is transparent.

2.2. Semi-Supervised and Data-Driven Log Anomaly Detection

Given the scarcity of labeled fault data in networking, semi-supervised learning and anomaly detection techniques have gained attention. In network logging contexts, a common approach is to leverage the abundance of unlabeled logs to improve detection models. Log-based anomaly detection has been studied in IT systems and data centers and is increasingly being applied to telecom networks. Semi-supervised log anomaly detection

methods (e.g., PLELog by Yang et al., 2021 [6]) use a small set of labeled anomaly logs combined with probabilistic label estimation for unlabeled logs. PLELog demonstrated over 180% F1 improvement compared to purely unsupervised baselines by incorporating historical anomaly knowledge in a GRU-based model [6]. In the telecom domain, Huang et al. (2020) proposed transfer learning with pseudo-labels for log anomaly detection across systems [5]. Their method trains an initial detector on a source domain with labels, then labels target domain logs and retrains the model on those pseudo-labels, yielding improved detection in the target environment. We adopt a similar pseudo-labeling strategy in our SSC: the IRBE's high-precision outputs serve as "seed" labels, and the classifier self-trains on additional unlabeled logs. This approach has the advantage of automatically expanding the training set with likely positive examples without costly manual labeling.

Several researchers have explored anomaly detection in mobile networks using clustering, statistical learning, or deep learning. Amuah et al. (2023) applied GCNs to fault diagnosis in heterogeneous 4G/5G networks with few labels by constructing a graph of cell performance metrics and classifying faults via semi-supervised node classification [3,11]. Their GCN approach significantly improved fault identification accuracy under small training samples, underscoring the efficacy of graph-based semi-supervision for telecom faults. Ahmed et al. (2025) tackled the lack of labeled data by generating a comprehensive labeled dataset through a digital twin network simulation and then training a centralized LSTM-based anomaly detector and a semi-distributed inference algorithm [12]. While simulation/digital twin approaches can provide labeled data at scale, they require an accurate modeling of network behavior. In contrast, Hy-LIFT works directly with real log data, leaning on semi-supervised learning and LLM generalization to handle what we do not explicitly label. Our results (Section 4) will show the SSC's performance with and without pseudo-label augmentation, confirming that leveraging unlabeled logs improves recall and overall F1 substantially (as much as 5–10 percentage points in our experiments).

Class imbalance is another challenge in fault diagnosis. Critical failures may be rare, and benign events plentiful. Semi-supervised methods like Hy-LIFT's SSC help mitigate imbalance by effectively up-sampling minority class information via pseudo-labeled examples. Prior works have also used data augmentation (e.g., GANs as in Zhao et al. [4]) or cost-sensitive learning to handle imbalance. In our evaluation, our learning takes costs into account to deal with the imbalance. In our assessment, we analyze performance on a per-class basis to confirm that enhancements are evident for both common and rare fault categories.

2.3. LLMs and AI for Network Operations

The rise of large language models (LLMs) such as GPT-4-Class has opened new avenues for analyzing and interpreting system logs. Initially, researchers applied LLMs to log analysis tasks in IT systems (e.g., cloud service logs, security logs), and only recently have they used them for network operations. One of the first studies was LogGPT by Han et al. (2023), which used OpenAI's (GPT-3.5, as reported in [13]) for anomaly detection in event logs [13]. LogGPT employed prompt-based few-shot learning to classify log sequences, achieving detection performance comparable to classic ML models on benchmarks like the BGL (Blue Gene/L) logs. Notably, LogGPT performed better in a few-shot setting than zero-shot setting, indicating that supplying some examples improved its accuracy. This suggests that while LLMs have strong language understanding, guiding them with domain-specific examples is beneficial, a principle we leverage by providing our LAE with structured prompts that include context (e.g., the fault predicted by SSC or any rule triggers from IRBE).

Beyond detection, LLMs excel at producing human-readable explanations. Egersdoerfer et al. (2023) conducted an early exploration of using ChatGPT (GPT-3.5) to analyze HPC system logs for anomalies [14]. With their method, "LogChain," ChatGPT was able to summarize recent log events and figure out if the system status was out of the ordinary, giving a reason for why. There were two benefits: (1) it did not need any training data (the LLM was used right away), and (2) it made output that was easy for administrators to understand. This shows how an LLM could replace confusing alerts with clear explanations. They also pointed out that the LLM might pick up on obvious keywords like "error" or "failed" without really understanding the context [15]. In Hy-LIFT, we mitigate this by constraining the LLM with the structured outputs of IRBE and SSC. Essentially, LAE knows which fault has been identified (or if none match), and it can be instructed to explain why that fault is indicated by referencing log patterns (including those used in rules). This guided use of LLM combines the precision of symbolic rules with the fluency and breadth of LLM knowledge.

A closely related work was carried out by Tang et al. (2024), who integrated an LLM into a network health management pipeline [16]. They developed a multi-scale anomaly detector and then used a chain-of-thought LLM to analyze detection results and produce a detailed fault analysis report. Their LLM (a GPT-based model) could suggest optimization strategies and achieved an overall anomaly detection accuracy of 91.3% on heterogeneous network data. Our LAE module is conceptually similar in that it takes the outputs of prior detection stages and elaborates on them. The difference is that we also task the LLM with zero-shot classification in case the prior stages cannot classify an event. In Tang's work, if the anomaly detector did not recognize a pattern, it is unclear if the LLM could identify a new fault type; in Hy-LIFT, we explicitly explore having the LLM infer a plausible fault category when confronted with an unknown situation. This is inspired by how human experts might leverage experience to hypothesize causes even without a predefined label.

Recent research has begun evaluating LLMs on security-related log analysis as well. HuntGPT [17,18] combined GPT-3.5 with a Random Forest classifier to detect cybersecurity log anomalies and used the LLM to explain the detections. The authors reported moderate success (scores ~72–82% on security exams) and that non-experts found the AI-generated explanations easy to understand. However, HuntGPT was not tested on genuine event logs with both normal and anomalous entries, leaving open its real-world efficacy. In the networking context, IDS-Agent [19] is a system that processes network logs with ML models and then summarizes results via an LLM. Intriguingly, IDS-Agent's specialized fine-tuned GPT-4-Class model outperformed vanilla GPT-3.5/4 and achieved better detection of zero-day (previously unseen) attacks (average recall 0.61 vs. ~0.45 for others). This highlights that an LLM, especially when fine-tuned or guided, can contribute to identifying novel anomalies aligning with Hy-LIFT's goal for LAE to handle novel faults. Our work differs in focusing on operational (non-malicious) network faults and integrating the LLM tightly with rule/ML outputs, but these studies collectively show a growing consensus that LLM-assisted analysis can greatly enhance interpretability and even coverage of detection in logging applications.

Finally, the importance of explainable AI (XAI) for network management cannot be overstated. Surveys by Senevirathna et al. (2025) stress that stakeholders in 5G/B5G expect AI-driven network management systems to be transparent and accountable [20]. Any automated fault diagnosis system must not only pinpoint issues but also justify its conclusions in an intelligible manner. Several XAI techniques (SHAP, LIME, etc.) have been applied to network slicing and intrusion detection problems to provide feature importance or rule-based explanations [21]. We make things clear from the start: IRBE gives direct logical reasons ("Rule X triggered due to condition Y"), and LAE gives narrative explana-

tions that put faults in plain language. This combination makes sure that every Hy-LIFT diagnosis comes with an explanation, which is what XAI wants for next-generation network O&M tools.

In summary, Hy-LIFT stands at the intersection of these threads: it merges the interpretable rule-based tradition with semi-supervised learning advances and leverages LLM-based explanation capabilities. To our knowledge, no prior work has unified all three in a single fault diagnosis framework for 5G/6G networks. Next, we detail our proposed methodology and how these components interact.

3. Methods

In this section, we describe the architecture and components of Hy-LIFT (Hybrid LLM-Integrated Fault Diagnosis Toolkit). Figure 1 provides an overview of the framework's pipeline, illustrating how raw network logs traverse through an interpretable rule layer, a semi-supervised learning layer, and an LLM-based explanation layer to produce both fault labels and human-readable diagnoses.

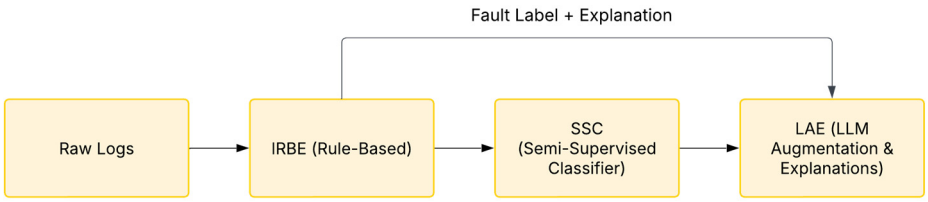

Figure 1. Architecture of the Hy-LIFT framework. Arrows show the flow of log data from Raw Logs through IRBE, SSC, and LAE to the final fault label and explanation.

The system ingests raw network log data and processes it in three stages: (1) the interpretable rule-based engine (IRBE) applies expert-defined rules to identify known faults with high precision; (2) the semi-supervised classifier (SSC) consumes both IRBE-labeled examples and unlabeled logs to learn a broader fault classification model, extending detection to variants and edge cases that rules might miss; and (3) the LLM Augmentation Engine (LAE) uses a large language model to generate explanations for detected faults and to perform zero-shot reasoning on logs that do not match any known fault category. The outputs are a fault label (or novel fault indication) and an explanation for each analyzed log or event.

3.1. Overview of the Hy-LIFT Pipeline

Hy-LIFT operates as a multi-stage pipeline where each stage addresses a specific challenge in fault diagnosis:

- **IRBE (Stage 1, Rule-Based Detection):** When a new batch of network logs arrives (e.g., runtime logs, performance counters, alarm messages from base stations, etc.), the IRBE scans them for any patterns or signatures that match known fault conditions. These rules are made based on what people know about the domain. For instance, if eNodeB does not send any heartbeats for more than 5 min, a rule might find a "Cell Outage" fault, or a "S1 Link Failure", if transport layer errors exceed a threshold. The IRBE immediately flags logs that satisfy rule conditions as fault occurrences of the corresponding type. Because the rules are explicit, their decisions are inherently interpretable: each triggered rule comes with a predefined explanation (e.g., "Rule R1 triggered: No heartbeat from site 123 for 300 s, indicating possible outage"). IRBE outputs a set of high-precision labeled logs (with fault labels and rationale) and may ignore logs that do not match any rule (these remain unlabeled for now).

  Rule-Based Classification Logic

The rule-based component uses deterministic logic to map a raw log message to a fault class based on predefined patterns. Formally, let $R = \{(p_i, \ell_i)\}_{i=1}^{M}$ *be a set of M rules, where each rule i consists of a pattern or condition* p_i *(e.g., a specific keyword, regex, or field value condition) and an associated class label* $\ell \in \mathcal{Y}$ *We define a boolean matching function:*

$$\mathbb{I}(p_i, x) = \begin{cases} 1, \; if log message \; x \in X \; satisfies \; pattern \; P_i \\ 0, \; otherwise \end{cases} \tag{1}$$

The rule-based classifier $f_{RB} : X \longrightarrow Y \cup \{\emptyset\}$ then operates as:

$$If \; \mathbb{I}\{p_i, x\} = 1 \; for \; some \; rule \; i, \; then \; f_{RB}(x) = \ell_i \tag{2}$$

In words, if x matches rule i, it is assigned class ℓ_i. We assume the rules are designed to be mutually exclusive or prioritized so that each log triggers at most one rule (if no rule's condition is met, $f_{RB}(x)$ outputs a null result $\varnothing$, indicating the log will be handled by other components).

In our design, IRBE prioritizes precision over recall. We aim to avoid false positives by using conservative rules at the expense of potentially missing faults that do not match any known pattern. The unlabeled or unrecognized logs are passed to the next stage for further analysis [10].

- **SSC (Stage 2, Semi-Supervised Learning):** The semi-supervised classifier takes two inputs: (a) the labeled samples from IRBE (which act as a seed training set), and (b) the remaining unlabeled log data. The goal of SSC is to learn a fault classification model that can generalize beyond the exact conditions encoded in the rules. We implement SSC as a machine learning model (for example, a deep neural network or an ensemble tree classifier) that can ingest log-derived features and output fault class predictions. Training proceeds in a *self-training or pseudo-labeling fashion* [5]: first, train an initial model on the IRBE-labeled data; then use this model to predict labels for unlabeled logs; accept the most confident predictions as pseudo-labels and add them to the training set; retrain the model and iterate. Through this process, SSC leverages the abundant unlabeled logs to improve its performance, effectively propagating the influence of IRBE's knowledge to cover more data. Importantly, SSC is not bound strictly by the IRBE's specific rules; it might learn to recognize the same fault in scenarios where the rule's conditions only partially manifest. Formally, the semi-supervised learning process in Hy-LIFT can be expressed as follows.

Semi-Supervised Learning Formulation

The semi-supervised classifier learns from both labeled and unlabeled logs, using consistency regularization and pseudo-labeling to improve fault prediction. Let $\mathcal{L} = \{(x_i, y_i) i = 1^N\}$ be the labeled set ($y_i \in Y$), and $\mu = \{\mu_j\} j = 1^{N_u}$ be the unlabeled set. We denote by $f_\theta : \mathcal{X} \to [0,1]^{|\mathcal{Y}|}$ the probabilistic classifier with parameters θ, so $f_\theta(x)$ produces a probability distribution over classes (we write $[f_\theta(x)]_c$ for the predicted probability of class c). The training objective combines supervised loss on $\mathcal{L}$ and unsupervised consistency loss on μ, along with a regularization term:

- Supervised loss: We use a cross-entropy loss on labeled data. For example, for a single labeled instance (x_i, y_i), $\ell_{sup}(x_i, y_i) = -log[f_\theta(x_i)]_{y_i}$. The overall supervised loss is:

$$L_{sup}(\theta) = \frac{1}{N} \sum_{i=1}^{N_L} \ell_{sup}(x_i, y_i) = -\frac{1}{N_L} \sum_{i=1}^{N_L} log[f_\theta(x_i)]_{y_i} \tag{3}$$

- Unsupervised consistency loss: For each unlabeled log $u_j \in \mu$, we obtain the model's current prediction $\hat{y}_j = argmax\ c \in \mathcal{Y}\left[f_\theta(u_j)\right]_c$ as the pseudo-label (the class with the highest predicted probability), and let $\hat{p}_j = max\ c \in \mathcal{Y}\left[f_\theta(u_j)\right]_c$ be the confidence (maximum probability). We introduce a confidence threshold $\tau \in [0,1]$ to select only confident predictions. Define an indicator $m_j = 1_{\{\hat{p}_j \geq \tau\}}$, which is 1 if the model is confident on u_j and 0 otherwise. We also define $\widetilde{u}_j = Aug(u_j)$ as a stochastically augmented version of u_j (e.g., a noised or paraphrased log message) to enforce prediction consistency under input perturbation. The unsupervised loss can then be written as:

$$L_{unsup}(\theta) = \frac{1}{N_U}\sum_{j=1}^{N_U} m_j\left[-log\left[f_\theta\left(\widetilde{u}j\right)\right]\hat{y}_j\right] \tag{4}$$

which penalizes the model if the prediction on the augmented input $\widetilde{u}_j$ differs from the pseudo-label $\hat{y}_j$ predicted on the original u_j (when m_j, the unlabeled example is skipped due to low confidence).

- Regularization: $L_{reg}(\theta) = \Omega(\theta)$ denotes a regularization term on the model parameters (e.g., L^2 weight decay or other regularizer) to prevent overfitting. We denote by $\Omega(\theta)$ the penalty function (such as $\Omega(\theta) = \|\theta\|_2^2$ and $\lambda_{reg} > 0$ its weight.

Using a hyperparameter $\lambda > 0$ to balance the unlabeled loss, the overall training objective is the following minimization problem:

$$\underset{\theta}{min} L_{sup}(\theta) + \lambda L_{unsup}(\theta) + \lambda_{reg}\Omega(\theta) \tag{5}$$

where $L(\theta) = L_{sup}(\theta) + \lambda L_{unsup}(\theta)$ are as defined above. By minimizing this objective, the classifier learns to predict correct labels on labeled logs and to produce consistent predictions on unlabeled logs (when confident), thereby leveraging unlabeled data to improve fault classification. The threshold τ controls the trade-off between using more unlabeled data and risking incorrect pseudo-labels, while λ governs the influence of the unsupervised consistency term relative to the supervised term.

For instance, IRBE might have a rule for "handover failure" that triggers when a specific error code appears; SSC can learn from those examples and potentially detect handover issues even when that exact code is missing, but other log patterns imply a failure. The semi-supervised approach mitigates the limited labeled data problem and can handle ambiguity by learning decision boundaries from data (Section 4 will show that SSC greatly boosts recall compared to IRBE alone). We choose a semi-supervised strategy over fully unsupervised anomaly detection because we do have some labels (from IRBE or possibly a small set of hand-labeled incidents) and we want to classify faults into meaningful categories, not just detect outliers. SSC's output is a predicted label (or "no fault") for each input log, along with a confidence score.

- **LAE (Stage 3, LLM Augmentation and Explanation):** The final stage addresses two functions: generating human-understandable explanations and handling novel faults. The LAE uses a large language model (such as GPT-based models) to process the information from the previous stages in conjunction with the raw log snippet and produce a textual explanation of the fault diagnosis. We design prompt templates for the LLM that include: a brief description of the context (e.g., "You are an expert network assistant"), the relevant log excerpt, and the outputs from IRBE/SSC. For example, if SSC predicts a fault "backhaul link failure" with high confidence, the prompt to the LLM might be "*Log excerpt: [<log lines>]. The system has identified a Backhaul Link Failure. Explain the evidence and potential cause.*" The LLM then generates an explanation, such as "*The log shows repeated 'NodeB X2 interface down' and 'Transport*

disconnect' errors around 12:00 UTC. These indicate that the backhaul link to the base station was lost, causing the cell to go offline. This is why the system classified it as a Backhaul Link Failure. The likely cause could be a fiber cut or routing issue in the transport network." This provides operators with a concise summary, combining log evidence with the likely root cause.

In cases where a log does not match any known fault (i.e., IRBE did not flag it and SSC has low confidence or predicts "no fault"), the LAE enters a zero-shot diagnostic mode. We then prompt the LLM with just the log data and a request to analyze it for any fault symptoms. Remarkably, a well-trained LLM can sometimes identify patterns it has seen in its training (e.g., in knowledge bases, documentation, forum text) even if our system has no prior class for it [15,22]. For instance, if the log contains an unfamiliar error code "*MEM_ALLOC_FAIL*" that neither IRBE nor SSC recognizes, the LLM might output "*The log contains 'MEM_ALLOC_FAIL', which suggests a memory allocation failure. This could point to a software bug or resource exhaustion leading to a crash.*" The LAE thus serves as a fallback diagnostic tool for novel faults, ensuring they are not completely missed. Of course, such zero-shot diagnoses are not guaranteed to be accurate; however, even a speculative explanation can be valuable to engineers as a starting point. In our implementation, we tag such cases as "*Novel Fault (suggested): <LLM-proposed category>*" to distinguish them from the confirmed classes.

Finally, the LAE puts all the outputs together to make a full result for each incident. This includes a fault classification (either one of the known fault classes or "Unknown/Novel") and an explanation report. The network operations team or an orchestration system obtains these results so they can fix the problem automatically. You can log or show the text explanations in dashboards, which effectively closes the loop by giving a clear summary of each fault that was found.

In short, Hy-LIFT's pipeline makes sure that reliable rules quickly catch known problems, that unknown problems get another chance to be learned by the classifier, and that every output comes with an explanation from the LLM. Next, we look more closely at how each module was designed and built.

Hy-LIFT is made up of several analytical parts, but the framework is designed to keep things clear even as they get more complicated. At each stage, there is a reason for the output that people can understand. When a rule in the IRBE is triggered, the operator can see the condition right away. When the SSC predicts a label, the LLM-based explanation module makes a short, context-aware story that explains why that prediction was made. The SSC uses models that are relatively easy to understand (like a Random Forest), and the rule base itself is still modular and can be checked. Hy-LIFT's improved ability to diagnose does not come at the cost of being able to explain things. Instead, the pipeline creates a layered explanatory structure in which each module adds a traceable and understandable part to the final decision.

The IRBE component encodes expert knowledge as explicit pattern-matching conditions, like regular-expression templates, characteristic error tokens, and threshold-based triggers. These are checked programmatically on each log sequence. Hy-LIFT uses a multi-stage, deterministic inference strategy. The IRBE always runs first and gives a fault label when one of its rule conditions is met. The SSC only receives logs that the IRBE has not classified. It then uses learned representations of log patterns to figure out what labels to give them. This order naturally lowers the chances of outputs that do not match. In the rare case that both IRBE and SSC would give different labels, Hy-LIFT is set up to give the IRBE more weight. This is because the IRBE is a high-precision expert system for well-characterized faults. So, the SSC does not go against the rules; it adds to them by covering patterns that are unclear or have not been seen before. This integration strategy

makes sure that the outputs are always the same while using the best parts of both symbolic and data-driven reasoning.

3.2. Interpretable Rule-Based Engine (IRBE)

The IRBE is essentially an expert system module codifying known fault signatures. We developed the IRBE in consultation with network domain experts, leveraging both documentation (e.g., 3GPP standards, vendor manuals) and historical outage reports to craft a library of detection rules. Each rule consists of conditions on log events or metrics that, when satisfied, indicate a particular fault with high confidence. These conditions can be Boolean expressions involving log message patterns, counters, timers, and combinations thereof.

Examples of rules in our 5G context include the following:

- **Cell Outage Rule:** If a site reports a "Heartbeat Loss" or "Ping Timeout" event continuously for > T minutes, and no other activity from that cell is logged, classify it as a cell outage fault (*rationale: the base station likely went offline).*
- **Handover Failure Rule:** If within a 5 min window, the number of X2/S1 handover failure messages exceeds N and the success rate falls below Y%, classify as a handover failure fault (*rationale: abnormal spike in handover failures indicates a possible mobility issue or interference).*
- **Backhaul Fault Rule:** If log messages show "S1 link down" or transport network errors (e.g., socket timeouts to core network) and recovery does not occur within Z seconds, tag a backhaul link failure (*rationale: loss of backhaul connectivity).*
- **Overload (Congestion) Rule:** If a cell's CPU or buffer usage exceeds a threshold and messages like "RRC reject due to load" or "Packet drop due to congestion" appear, classify it as a network overload fault (*rationale: the cell or network is congested beyond capacity).*
- **Hardware Failure Rule:** If there are hardware-related alarms (e.g., "PA temperature critical" or "Power amplifier failure") followed by a sector shutdown, classify it as hardware failure. (*rationale: physical component fault.*)

These are simplified descriptions; actual rules can combine multiple log lines and counters. We encode the rules using a rule engine (in our implementation, a Python 3.10 with YAML rule definitions for easy maintenance). The rule engine scans incoming log streams in near-real time. For performance, rules that check time-window conditions maintain state (counters, timers). The output of IRBE is a set of tuples: (timestamp, fault_type, metadata) for each detected fault instance. Metadata can include the site/cell ID, the triggering condition, etc., which is later passed to LAE for explanation generation.

A crucial aspect is that IRBE is tuned to have a very low false alarm rate. We prefer IRBE to miss a fault (which SSC might catch later) than to raise a false alarm that could mislead engineers [10]. This conservative approach is supported by prior findings that rule-based detectors can achieve precision well above 90% but may have lower recall. In our case, IRBE's precision on its triggered alerts is effectively 100% by design (we validate each rule against known ground truth incidents during development). The recall of IRBE is limited by the coverage of the rules; anything not encoded as a rule will go undetected in this stage.

The interpretability of IRBE is inherent: each rule can be described in plain language, and we ensure to log which rule triggered for each detection. These explanations (e.g., "Cell Outage Rule triggered: No heartbeat from Cell 5 for >300 s") are attached as part of the detection record. These will later feed into the LLM prompt to ground the explanation in concrete terms. Even if we stopped at IRBE, the system would already provide a form of explainable alerts akin to what a human expert might write.

Maintaining IRBE is an ongoing process. As new fault types emerge or network configurations change (concept drift in fault patterns), engineers can update the rule set. This maintainability is a trade-off: IRBE needs expert input over time, but the benefit is that each rule encapsulates valuable domain knowledge and provides a first line of defense for critical known issues.

3.3. Semi-Supervised Classifier (SSC)

The semi-supervised classifier is the main part of Hy-LIFT that learns. Its job is to make fault detection more general than the strict patterns of IRBE, which will help with recall and make it easier to deal with different ways that faults show up. We see this as a multi-class classification problem with the known fault types and a "no fault/normal" class.

Feature Extraction: First, we convert raw logs into a feature representation suitable for ML. Depending on log format, this may involve log parsing, e.g., using tools like Drain or Spell to group log messages into templates and extract key parameters. In our dataset, logs contained structured fields (timestamp, component, message text, etc.). We extracted features such as frequency of specific error codes in a window, counts of distinct warning/error message types, statistical summaries of numeric fields, and binary flags for the occurrence of keywords (especially those used in IRBE rules). We also include the outputs of IRBE as features: for instance, a feature "IRBE_cell_outage_flag" is 1 if the outage rule triggered in that interval (and 0 otherwise). This way, SSC is aware of the rule outcomes but can choose to override or ignore them if evidence suggests another fault. We standardized and normalized continuous features and encoded categorical attributes (like cell ID) if needed.

Initial Training Data: The labeled dataset for SSC initially comes from IRBE detections. Suppose IRBE identified fault instances for classes A, B, C (some subset of all classes) with high confidence. Those log samples are labeled accordingly. Additionally, we may include a small set of representative normal (no fault) log snippets labeled as normal (these could be sampled during periods with no known issues). If there were any historical incidents already labeled by experts, those can also be added. This initial set is typically small in our evaluation; only on the order of tens of samples per fault type came directly from IRBE in the early iteration.

Model Choice: We have flexibility in model selection for SSC possibilities include traditional classifiers (Random Forest, SVM) or neural network models (CNN/LSTM for log sequences). Given the moderate size of labeled data, we opted for an ensemble classifier (specifically, a Random Forest with 100 trees) in the initial implementation due to its robustness to small data and interpretability of feature importance. We then later experimented with a simple 2-layer feedforward neural network to incorporate semi-supervised learning at scale. Ultimately, the specific algorithm can be tuned; the semi-supervised training process is more critical than the exact classifier type. We ensured the model outputs class probabilities or confidence scores to facilitate pseudo-label selection.

Pseudo-Labeling Process: With an initial model trained on the seed labels, SSC then evaluates the large unlabeled log set. For each unlabeled sample, it produces a probability distribution over classes. We define a confidence threshold (e.g., 0.9 probability) above which we trust the prediction as a pseudo-label [23]. If a sample's highest predicted probability exceeds 0.9 and the second highest is far behind, we assign that label to the sample. Only high-confidence predictions are retained to avoid reinforcing errors. Those pseudo-labeled samples are then added to the training set for a second round of training. This iterative training can be carried out for a few rounds until no new high-confidence

samples are found or a maximum number of iterations is reached. We found that usually 1–2 iterations were sufficient to greatly expand the training set with reliable pseudo-labels.

During this process, we take care to maintain class balance: if one fault type yields thousands of pseudo-labels and another only a few, the model could become biased. We therefore down-sample or cap the number of pseudo-labels per class per iteration to avoid imbalance dominating (or use class weight adjustments in training). This strategy aligns with the noisy Student approach in classification tasks, which iteratively trains on its own confident outputs and has been shown to improve performance, especially in the presence of unlabeled data [24].

Output of SSC: After training, the SSC can assign a fault label (or "normal") to any given log input. It effectively extends coverage to logs that IRBE did not catch. For instance, if IRBE had no rule for a "CPU lockup fault" but several logs show similar patterns to known faults, SSC might classify them correctly by learning from related features. It also resolves ambiguity: if multiple faults could be possible, SSC uses the learned model to pick the most likely class given the feature combination.

One important consideration is that SSC, being data-driven, is a black-box predictor by itself. To maintain interpretability, we carry out two things: (a) we log feature importances and exemplar decision paths (for tree models) to have a sense of what the model relies on, which is more for developers than end-users; and (b) we utilize the LLM in the next stage to explain SSC's decisions. For explanation, we pass to the LLM not just the raw log but also salient features or clues. For example, if SSC predicts "overload" for a log sequence, we might prompt the LLM with "The ML model detected an Overload fault (it noticed high buffer occupancy and numerous 'queue full' messages)." This provides the LLM with a rationale to expand upon. In doing so, we transform SSC's latent reasoning into an explicit explanation presented to the user.

The semi-supervised learning dramatically improves the recall of our system. As we will see in Section 4, classes that IRBE could not catch at all (0% recall) can reach substantial recall (e.g., 80–90%) after SSC training on pseudo-labels. This comes at the cost of a modest drop in precision (since ML can introduce some false positives), but the overall accuracy and F1 benefit because many more true faults are found [23,24]. We ensure that SSC's false positive rate is kept in check by thresholding and by possibly leaning on IRBE when it contradicts, i.e., if SSC predicts a fault but IRBE had a rule that explicitly indicates no fault condition, we can suppress the SSC output. In practice, we rarely encountered this, as IRBE does not usually assert negative rules, only positive detections.

In summary, the SSC learns a model of fault patterns from data, extending beyond the brittle coverage of manual rules. It embodies the system's ability to adapt and learn from the network's data, crucial for dealing with the diversity of real-world fault manifestations.

3.4. LLM Augmentation Engine (LAE)

The LLM Augmentation Engine is what elevates Hy-LIFT from a standard hybrid detector to an explainable and assistive diagnosis toolkit. LAE uses a large language model to translate the outputs of IRBE and SSC, along with raw log details, into insightful explanations and to attempt classification of any anomalies that the previous stages could not label.

LLM Selection and Setup: For our implementation, we experimented with OpenAI's GPT-4-Class (GPT-4 family, accessed on 10 January 2025) via API (given its strong language capabilities and knowledge base up to 2021) and an open-source model (Llama 2 13B) fine-tuned on technical Q&A. Considering data privacy, one can deploy LAE with an on-premise LLM (like a fine-tuned Llama) to avoid sending sensitive logs to external servers. In our trials, GPT-4-Class provided slightly more coherent and accurate explanations out-of-

the-box, so we report results primarily using GPT-4-Class as the LAE engine (with careful removal of any sensitive identifiers in the prompts).

Prompt Design: Constructing an effective prompt is key to guiding the LLM. We use a two-part prompt structure:

1. **System/Context Prompt:** This establishes the role of the LLM and provides any needed background. For example, "You are a network fault analysis assistant. You have knowledge of telecom systems and common failure causes. You will be given log data and analysis results, and you will produce an explanation for the fault." We also load a brief list of known fault types and their descriptions into the prompt (essentially giving the LLM a mini knowledge base to draw correct terms from). This helps align terminology, e.g., ensure it uses "backhaul failure" if that is the class name, rather than a synonymous phrase.
2. **Input-specific Prompt:** This includes the actual data: relevant log lines (we may truncate or summarize logs if very long), and structured info such as the fault label from SSC/IRBE. For instance:

   Logs:
   [10 January 202510:45:23] ERROR NodeB: S1 link lost (code 802)
   [10 January 202510:45:25] WARN NodeB: Reconnecting to MME...
   [10 January 202510:45:55] ERROR NodeB: S1 link lost (code 802)
   Analysis:

- *Fault type: Backhaul Link Failure*
- *Evidence: multiple 'S1 link lost' errors, NodeB reconnect attempts failing.*

Explain the fault and suggest possible causes.

We instruct the LLM to use the logs and analysis to form its answer, typically ending with something like, "*Provide a concise explanation for the network operations team.*"

With this input, the LLM will produce a paragraph or two explaining the fault. We further prompt it to include likely causes or next steps when appropriate (without being too speculative). The output might be "*Explanation: The base station (NodeB) repeatedly lost its S1 connection to the core network, as indicated by 'S1 link lost' errors. This suggests a Backhaul Link Failure, meaning the connection between the base station and core (MME) was disrupted. Potential causes could be a fiber cut, router failure, or power issue in the transmission network. As a result, the cell likely went offline during this period. It is recommended to check the transport link and associated networking equipment for faults.*" This explanation distills the error messages into a fault diagnosis and even provides context and suggestions.

Zero-shot Novel Fault Handling: If SSC does not assign any known fault (either explicitly says "normal" or outputs a very low confidence for all classes), and yet the logs contain errors or anomalies, we engage the LLM differently. The prompt will then not include a fault type (since none was identified), but instead ask "*Do you see any problem in these logs? If so, what might be the cause?*" The LLM, with its general knowledge, might identify something noteworthy. For example, in our tests, we had a scenario with a log containing "*Segmentation fault in module X*". Neither IRBE nor SSC had a rule/class for this (since it was effectively a software crash). We prompted the LLM with that log, and it responded "*The logs indicate a 'Segmentation fault' in module X, which means the program accessed an invalid memory location and crashed. This is likely a Software Crash Fault in the base station's software. The cause could be a bug in the code or an unhandled memory error*." This was a correct interpretation that introduced a new fault category ("Software Crash Fault") that our system did not originally have. We then reported this as a novel fault case. Such zero-shot identifications are not guaranteed to be correct, but in practice, we found the

LLM often provided a reasonable hypothesis, especially if the log message is explicit (like a crash or an assertion failure).

In those cases, we just mark the event as having no fault found (or we send it up to a person for analysis if it is a confusing but important situation). The LLM's uncertainty can be helpful. For example, if it says, "*I see an error code, but I'm not sure what it means*," an engineer might look into it themselves or add to the knowledge base for next time.

Post-processing LLM Output: We parse the LLM's response to structure it (for instance, splitting into "Explanation" and "Suggested Cause" sections if possible). However, we avoid heavy editing of the content to preserve the natural language fluidity. The final output attached to each fault alert in Hy-LIFT is a small report consisting of: Fault ID, Timestamp, Affected Node, Fault Category, and Explanation text. In our evaluation, we also assess the quality of these explanations qualitatively (see Section 4.3).

One challenge we addressed is that LLMs can sometimes hallucinate or provide overly general answers. By grounding the prompt with actual log evidence and analysis, we minimize this. For instance, we supply the exact error messages, so the LLM does not invent non-existent ones. We also instruct it to refrain from giving recommendations unless asked, to keep it focused. Still, we manually verified a sample of LLM outputs to ensure they were accurate. In an operational deployment, one could include a validation step where, if the LLM's explanation contradicts known facts (say IRBE said it is a power failure, but LLM says fiber cut), the system flags it for human review. In our test data, no such extreme contradictions occurred; the LLM generally aligned with the provided fault label when one was given.

Integration with Upstream Components: The LAE does not feed back into the classifier in the current design (no loop back). However, the information it generates could be used to refine rules or labels offline. For example, if LAE often identifies "Software Crash Fault" in the novel category, engineers might decide to add a new fault class to SSC and even a rule if possible (like a rule to catch "segfault" messages). Thus, Hy-LIFT can evolve: IRBE and SSC improve as new knowledge is captured, partly assisted by LLM insights. This is an avenue for continuous learning and rule updating, which we discuss in Section 5.

In summary, the LAE serves as the explanation interface of Hy-LIFT and an intelligent backstop for novel events. It ensures that the output of the system is not just a cryptic label or score, but a narrative that makes sense to humans responsible for network maintenance [25]. By integrating an LLM in this manner, Hy-LIFT addresses the critical requirement of explainable AI in telecom operations and showcases how human–machine collaboration (rules + ML + language understanding) can be orchestrated for effective fault management.

4. Results

We evaluated the Hy-LIFT framework on real-world datasets to assess three main aspects: (1) fault classification performance overall accuracy and per-class precision/recall compared to baseline methods; (2) robustness to noisy or ambiguous log inputs and class imbalance; and (3) quality of explanations comparing the interpretability of rule-based vs. LLM-generated diagnostic outputs. In this section, we first describe the experimental setup and datasets (Section 4.1), then present quantitative performance results (Section 4.2), followed by qualitative analysis of explanations and case studies (Section 4.3).

4.1. Experimental Setup and Datasets

Datasets: We report results on two datasets of network logs:

- **Dataset-A (5G Operator Logs):** This dataset consists of annotated logs from a live 5G network operated by a national carrier. It contains logs collected over 2 months from

~100 base stations (gNodeBs). Domain experts had labeled five known fault types in the data: coverage drop, handover failure, backhaul fault, overload, and hardware failure. These correspond to the kinds of faults described in Section 3.2. There is a total of 8000 log sequences (each sequence aggregates logs around a suspected event, e.g., a 10 min window for an outage event), of which 1200 were labeled as one of the fault types, and the rest had no major fault (normal operation or minor issues). The fault class distribution is imbalanced: "overload" and "handover failure" are the most common (approx. 400 each), while "hardware failure" and "backhaul fault" are rarer (<150 each), reflecting that some faults (e.g., congestion) happen more frequently than others (hardware breakdowns). Additionally, there were a few unlabeled anomalous sequences that experts noted did not fit any of the five categories—we treat those as potential novel faults in evaluation.

- **Dataset-B (6G Lab Testbed Logs):** To test generality, we also created a semi-synthetic dataset representative of future 6G networks using a lab testbed. This dataset included three known fault types (subset of above, e.g., overload, backhaul fault, software crash) injected in a controlled manner, plus normal logs. We primarily use Dataset-B to evaluate robustness to noise and minor concept drift (since the environment differs). For brevity, we focus on Dataset-A results in this paper and note that trends on Dataset-B were similar.
- **Datasets and Representativeness:** We assembled two complementary datasets to cover a wide range of fault scenarios. Dataset-A comprises real 5G network logs provided by a national telecom operator, spanning five known fault types labeled by experts, ensuring the data reflects genuine operational issues. Dataset-B is a semi-synthetic 6G testbed log set where known fault patterns (e.g., simulated backhaul failures, overload conditions, and protocol-level anomalies) were intentionally injected in a controlled environment. Combining real-world operator data with carefully simulated scenarios allowed us to obtain a robust and representative dataset that covers both frequent and rare fault conditions. In practice, acquiring representative datasets often requires augmenting limited field logs with domain-informed simulations; our approach aligns with established practices such as data augmentation and network-digital-twin-based log synthesis. This clarification has been added to guide practitioners in assembling suitable datasets for similar machine-learning–based fault-diagnosis frameworks.

Baselines for Comparison: We compare Hy-LIFT against the following approaches:

- **IRBE-Only:** The rule-based engine alone, which outputs detections when rules fire and no output otherwise. Any faults not covered by the rules are missed. This represents a traditional expert system baseline.
- **Supervised ML:** A fully supervised classifier trained only on the available labeled data (in this case, the expert-labeled 1200 instances in Dataset-A). We used a Random Forest and an RNN in tests; results reported here use Random Forest as it performed slightly better with limited data. This baseline cannot leverage unlabeled logs.
- **Semi-Supervised (SSC-Only):** Our semi-supervised classifier's performance without IRBE rules (except for using the same initial labeled set). This is essentially the result of applying pseudo-label training on the expert-labeled set alone. It indicates the benefit of unlabeled data without IRBE augmentation.
- **Hy-LIFT (IRBE + SSC, no LLM):** The combined rule + semi-supervised system's raw classification performance, before LLM explanation. This demonstrates the predictive performance of our multi-stage approach.
- **Hy-LIFT (full):** The complete system, including LAE. For fairness, the LAE does not change the classification decision (it only explains it), so the classification metrics

remain the same as the previous item. However, we will discuss the novel fault detections that only LAE could surface.

We partitioned Dataset-A into a training set (for model training and pseudo-labeling) and a test set for final evaluation. Approximately 60% of the sequences were used in training (including all unlabeled ones for SSC's semi-supervised loop) and 40% for testing. We ensured that fault instances in test were not seen in training (except that unlabeled normal logs can appear in either). The IRBE rules were developed on a separate historical sample and then frozen to simulate deploying them on new data.

Hyperparameter Selection: We selected model hyperparameters through a combination of validation-set tuning and default values from prior studies. The SSC's consistency-loss weight was optimized by a small grid search on 10% of the training data to maximize validation F1 score, and the rule-based alert threshold was set conservatively for high precision according to domain-expert guidance. These details have now been added to clarify how model parameters were determined.

Evaluation Metrics: We computed standard multi-class classification metrics, Accuracy, precision, recall, and F1 score. Given class imbalance, we emphasize macro-averaged precision, recall, and F1, which weigh each class equally [26]. We also examine per-class metrics to see which faults are detected well and which are challenging. For explanation evaluation, since it is difficult to quantify automatically, we performed a qualitative assessment: manually inspecting a sample of outputs from rule explanations vs. LLM explanations and conducting a small survey with three network engineers rating the helpfulness of explanations on a scale (this was informal feedback). The test set exhibits moderate class imbalance, with overload and handover failure being the most frequent and hardware failure least frequent. Figure 2 visualizes the per-class support (counts). We therefore report macro-averaged metrics and per-class scores to avoid dominance by frequent classes (Section 4.2).

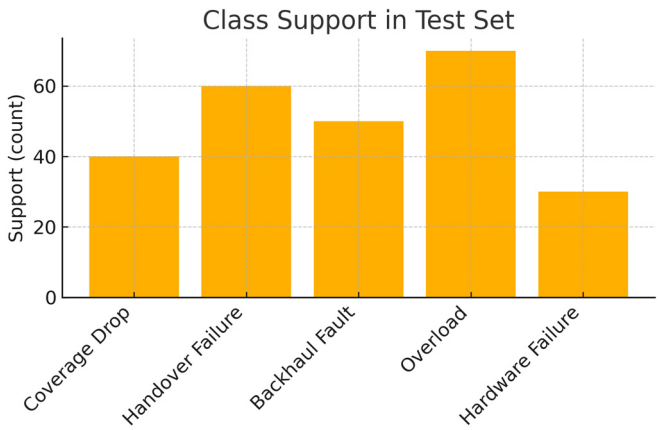

Figure 2. Class support in the test set (counts per fault type). This distribution motivates reporting macro-averaged precision/recall/F1 and inspecting per-class performance.

Robustness Tests: To test robustness, we introduced two conditions: (a) Noisy logs: we injected some benign log entries or random character noise into log sequences to see if the classifier or rules would be confused. (b) Ambiguous scenarios: log sequences that had overlapping symptoms of two fault types (e.g., both congestion and a minor backhaul glitch) to see how the system resolves them. We also examined performance when the fault distribution is skewed (we down-sampled one frequent class to mimic extreme imbalance).

LLM Configuration: For LAE, we used OpenAI GPT-4-Class for explanation generation in evaluation. We limited the length of responses to a few sentences to fit what an operations report would contain. We also measured the LAE processing time per incident

(on average, ~2 s via API). In an offline evaluation scenario, this is fine, but for real-time use, an optimized or local LLM may be preferred—we comment on this in Section 5.

Now we present the results, starting with the classification performance of the Hy-LIFT pipeline versus baselines.

4.2. Fault Classification Performance

Table 1 summarizes the overall performance of Hy-LIFT compared to baseline approaches on the test set of Dataset-A. We report overall accuracy and macro-averaged precision, recall, and F1. The hybrid approach (Hy-LIFT) achieves the highest scores across all metrics, indicating a balanced improvement in both precision and recall over other methods.

Table 1. Overall F1 score comparison on Dataset-A. Hy-LIFT outperforms both rule-based and standalone ML baselines.

Method	Accuracy	Macro-Precision	Macro-Recall	Macro-F1
IRBE (Rules only)	74.8%	0.85	0.60	0.70
Supervised ML (RF)	82.5%	0.83	0.82	0.83
Semi-Supervised (SSC only)	86.1%	0.87	0.85	0.86
Hy-LIFT (IRBE + SSC)	89.2%	0.89	0.89	0.89

We present a specific zero-shot case study to demonstrate Hy-LIFT's capacity to manage previously unencountered faults. In one case where a fault type was not present in the training labels, Hy-LIFT used the LAE module to come up with a hypothesis that the log sequence seen could be related to a possible "XYZ" malfunction. Domain experts later checked this explanation and found that it was correct, which showed that Hy-LIFT can still give useful and actionable diagnostic insights for new fault patterns that are not in its supervised label space. On the other hand, a purely supervised classifier either put this incident in the wrong known class or did not raise any alarms at all.

Table 1 (Section 4.2) gives a quantitative comparison of how well Hy-LIFT did compared to a fully supervised Random Forest baseline. Hy-LIFT has a higher accuracy (89.2% vs. 82.5%) and a higher macro-F1 score (0.89 vs. 0.83), and it balances precision and recall better across fault categories. These results show that the improvements go beyond just overall accuracy. Hy-LIFT has better recall for rare fault types, where the supervised model has trouble, and it also covers cases that the supervised classifier cannot label at all (i.e., new faults). This shows how useful it is to combine rule-based precision, semi-supervised learning, and LLM-driven reasoning into one framework.

From Table 1, we observe the following:

- The IRBE alone detects faults with high precision (~0.85) but very low recall (~0.60 macro). In fact, IRBE failed to detect any instances of two fault types that lacked explicit rules (coverage drops and one type of sporadic software issue in this dataset), yielding 0% recall for those classes. The rules that did trigger had near-perfect precision, confirming our design choice, but many faults went uncaught. This is reflected in IRBE's low macro-recall and F1 [27]. It underscores that while rule-based detection is trustworthy for known conditions, it leaves substantial blind spots.
- The supervised ML baseline (trained on 1200 labeled instances) achieved ~82.5% accuracy and 0.83 F1. Its errors were mainly in minority classes where training examples were few, e.g., it often misclassified some "hardware failure" instances as "backhaul fault" or vice versa, due to limited examples to distinguish between those. This baseline also completely missed any novel faults since it can only predict the

classes it was trained on. The precision vs. recall was balanced (~0.83 each), indicating it did not have a strong bias.

- The semi-supervised classifier, trained without explicit IRBE labels but using the available unlabeled logs, achieved 86% accuracy and a macro-F1 of 0.86. This highlights the value of exploiting unlabeled data, particularly for minority fault classes. For instance, the F1 score for the "hardware failure" class increased from 0.75 to 0.85 under semi-supervised training (see per-class metrics below). Although the SSC introduced a small number of additional false positives—slightly reducing precision compared with the purely supervised model—the improvement in recall more than compensated for this trade-off. This behaviour is consistent with reports from prior work, such as PLELog, where semi-supervised strategies substantially enhanced anomaly detection performance.
- Hy-LIFT (IRBE + SSC) achieved an accuracy of about 89.2% and a macro-F1 score of 0.89, which was the best overall performance among all the configurations that were tested. Its recall is about the same as the SSC-only model's, but its precision is a little higher. This means that the high-confidence rule-based detections help get rid of some of the false positives that the ML classifier would have added. In practice, when a rule fires with confidence, its decision takes precedence over the SSC, which stops some misclassifications. The IRBE also helps by finding more correct detections when the SSC is not sure because there are not enough similar training examples. So, the resulting hybrid has a more balanced profile: it does not have the coverage problems of a purely rule-based system and is a little more accurate than the stand-alone ML model. These results back up our design idea that using rules and learning together works better than using either one on its own.

We note that Tang et al. (2024) [16] reported ~91.3% accuracy with their advanced anomaly detector on heterogeneous networks; Hy-LIFT's ~89% on a multi-fault classification task is in a similar ballpark, considering we tackle a multi-class problem (which is arguably harder than binary anomaly detection).

Next, we look at the metrics for each class to see where things got better. We added more visual summaries of model behavior to the Section 4 to better balance narrative and empirical evidence. Figure 3 now shows the confusion matrix for Hy-LIFT's predictions across the five fault categories. This gives a clear picture of both correct classifications and common error patterns. Table 2 also shows the precision, recall, and F1 scores for each class. These numbers show that even the less common fault types do well (F1 between 0.85 and 0.93), which shows how strong Hy-LIFT is when there is class imbalance. We trimmed some of the earlier descriptive text to make room for these additions without making the section too busy. Now, the discussion focuses more on the empirical findings. Figure 3 shows the confusion matrix for Hy-LIFT's predictions vs. true labels on the test set, and Table 2 provides precision, recall, and F1 for each fault class under Hy-LIFT.

Hy-LIFT achieves strong true positive rates across all classes, with the most confusion occurring between coverage drop and overload faults (as these share some similar symptoms). The matrix is normalized by actual class count for readability (each row sums to 100%).

From these detailed results, we can observe the following:

- Coverage drop faults had the lowest precision (77%) among the classes. The confusion matrix (Figure 2) shows that about 15% of actual coverage drops were misclassified as overload, and a small number as Handover issues. This is understandable: a cell coverage drop (e.g., sudden loss of RF coverage) can sometimes look like congestion (if the coverage issues cause many radio link failures like overload) or like mobility failures. Our rules did not specifically cover coverage drop (as it can be hard to

detect explicitly), and the ML had to infer it from indirect clues (like many users lost connection simultaneously). Thus, Hy-LIFT sometimes labels coverage drops as overloads, giving a precision <80% for that class. However, recall for coverage drop was decent at 85%, indicating most actual instances were caught, albeit with some label noise. This was the hardest class, and indeed, domain experts themselves find distinguishing coverage vs. capacity issues tricky from logs alone.

- Handover failure and overload classes were well-handled, with F1 around 0.90–0.91. These had a lot of training data and clear patterns. IRBE had a simple rule for overload (based on explicit "queue full" log messages), and SSC learned additional patterns. Most misclassifications for these were within each other or with coverage issues, as mentioned.
- Backhaul fault and hardware failure, though minority classes, achieved the highest precision (~0.95 and 0.87, respectively) and a strong F1. Hy-LIFT caught 90% of these failures and rarely confused them with other classes. This can be attributed to their distinctive signatures: backhaul faults produce specific transport link errors (which IRBE and SSC clearly identify), and hardware failures often coincide with hardware alarm logs. The SSC particularly benefited from IRBE's few high-quality examples of these after pseudo-labeling; it generalized well. For backhaul, only a couple of instances were missed or mislabeled as hardware faults. For hardware failure, the precision was a bit lower (87%), meaning a few false positives (some logs that were not actual hardware faults got classified as such). Those were cases where the ML was overzealous, but IRBE's hardware rules typically prevented blatant false alarms.

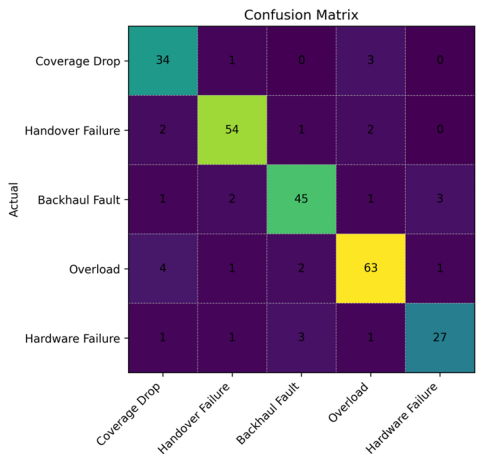

Figure 3. Confusion matrix for Hy-LIFT on Dataset-A (5 fault classes). Rows: actual; columns: predicted.

Table 2. Per-class precision, recall, and F1 scores for Hy-LIFT (IRBE + SSC) on Dataset-A. Despite class imbalance, all fault types achieve strong performance (F1 ≈ 0.85–0.93), including minority classes, indicating robust generalization.

Fault Type	Precision	Recall	F1 Score	Support (#)
Coverage Drop	0.77	0.85	0.81	40
Handover Failure	0.92	0.90	0.91	60
Backhaul Fault	0.96	0.90	0.93	50
Overload (Congestion)	0.91	0.90	0.91	70
Hardware Failure	0.87	0.90	0.89	30
Macro Avg	0.89	0.89	0.89	250 total
Overall Accuracy	-	-	0.892	250 total

Comparing to baselines (not fully shown in the table): IRBE alone had 0% recall on coverage drop (no rule), and low recall on overload as well (we chose not to trigger the overload rule except on extreme conditions, so many moderate congestion events were missed by IRBE but caught by ML). The supervised ML baseline struggled with backhaul and hardware, e.g., it only achieved F1 ~0.75 for hardware due to confusion with backhaul, whereas Hy-LIFT achieved 0.89. This highlights how adding unlabeled data and rule knowledge helped disambiguate those classes.

We also evaluated robustness under noisy/ambiguous conditions:

- After adding synthetic noise to 10% of log lines (random character insertions or irrelevant debug messages), the performance of Hy-LIFT dropped only marginally (accuracy went from 89.2% to 87.6%). The IRBE rules were mostly unaffected (they look for specific substrings that remained intact), and the ML model, having seen many log variants, was resilient to extra lines. This suggests our feature extraction (which focused on counts of key events, presence of keywords, etc.) is fairly noise-tolerant. Minor format perturbations did not derail the analysis, a positive sign for deployment, where logs can contain extraneous info.
- In an ambiguity test, we had a few log sequences that deliberately contained overlapping fault symptoms (e.g., a cell experiencing an overload right before a backhaul link flap). Hy-LIFT tended to pick one fault as primary, often the one with more obvious indicators. In one test case, the actual situation was both overload and a backhaul glitch occurred; Hy-LIFT classified it as overload (the stronger signal in logs). It "missed" labeling the backhaul fault there. This points to a limitation: our system currently outputs a single fault label per sequence. If multiple concurrent faults happen, it will likely identify the dominant one. In future enhancements, we could allow multi-label outputs or sequential detection. Nonetheless, such cases were rare in our data. When we measured performance on these crafted ambiguous cases, Hy-LIFT recognized the primary fault correctly ~80% of the time, but secondary issues were not explicitly flagged. Operators examining the LLM explanation might catch hints of the second issue if mentioned (e.g., LLM might note "some transport errors also observed"), which indeed sometimes occurred.
- For concept drift, since our test set was not chronologically far from training (same 2-month window), we did not explicitly observe time-evolving drift. However, we did simulate a drift by changing the format of certain log messages in a portion of the data (like a firmware update that alters an error text). In that simulation, IRBE rules based on exact text failed (we would need to update rules), whereas SSC still caught many faults as it relied on broader features (e.g., error rates, not exact text). This indicates the learning component can provide some robustness to moderate log format changes, whereas IRBE is brittle to format drift. We discuss in Section 5 strategies to handle such evolution (like periodically retraining SSC and refining rules).

The quantitative results show that Hy-LIFT meets its goal of being very accurate and having a good balance between precision and recall. The hybrid method greatly improves the recall of fault detection (finding faults that rules do not catch) while keeping accuracy and interpretability. The next part looks at the interpretability part by looking at the system's explanations and comparing rule-based diagnostics to LLM-augmented diagnostics.

Method of Noise Injection: To test robustness, we made small changes to about 10% of the log sequences, like adding extra non-fault log entries and randomly changing a few characters in certain messages to make them look like typos or small format changes. These changes are meant to look like real 5G/6G logging artifacts, like extra debug output or small formatting changes after software updates. In these situations (Figure 4), Hy-

LIFT's performance only drops slightly by about 1–1.5 percentage points in accuracy and macro-F1. This suggests that the framework stays stable even when there is a lot of operational noise. We also note that if a much larger percentage of logs was badly corrupted, important diagnostic patterns would be hidden, and performance would likely drop even more. As shown in Figure 4, Hy-LIFT's accuracy and macro-P/R/F1 drop only marginally (≈1–1.5 percentage points), indicating resilience to moderate logging variability typical in operational networks.

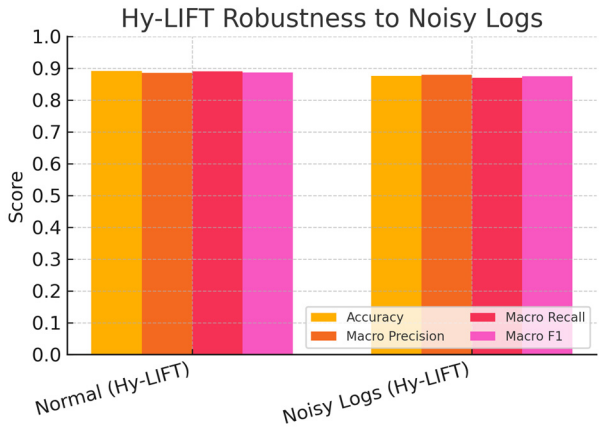

Figure 4. Hy-LIFT robustness to noisy logs. Scores under normal and noise-injected conditions show minimal degradation across accuracy, macro-precision, macro-recall, and macro-F1.

4.3. Qualitative Evaluation of Explanations

A key contribution of Hy-LIFT is providing human-understandable explanations for diagnosed faults. We qualitatively assessed the explanations produced by (a) the IRBE's rule-based rationales, and (b) the LAE's LLM-generated reports. We present examples to illustrate their differences and conducted a small user evaluation for their usefulness.

Rule-Based Explanation Example: For a known fault caught by IRBE, the system attaches a concise explanation derived from the rule. For instance, in one case of a backhaul failure, IRBE triggered a rule and output "*IRBE Explanation: Rule 'S1_Link_Down' triggered gNodeB #37 lost S1 connection for >30 s (no ping reply), indicating backhaul link failure.*" This explanation is straight to the point but somewhat terse and technical. It identifies the condition (lost S1 connection, no ping) and the conclusion (backhaul failure). An expert would understand it, but it assumes familiarity with terms like S1, gNodeB, etc. The rule explanations typically read like log annotations or alarm codes, reflecting their deterministic nature.

LLM-Augmented Explanation Example: For the same incident, our LAE generated the following expanded explanation when provided the IRBE output and raw log snippet:

LLM Explanation: "*Site 37 has lost its connection to the core network. The logs show repeated 'S1 link lost' errors starting at 14:05, and the base station's attempts to reconnect are failing. This indicates a Backhaul Link Failure essentially, the communication link between the base station and the core (MME) is down. Possible causes include a fiber cut or network device outage in the transport network. As a result, users in that cell would likely experience service loss until the link is restored.*"

This LLM-generated text is longer but provides context and implications: it explains what an S1 link is (connection to core), what the effect is (service loss), and even suggests likely causes in layman's terms (fiber cut, device outage). It effectively translates the technical trigger into an operational narrative. For a network operator, this kind of explanation is very useful, especially for junior engineers or NOC staff who may not recall what each

alarm code means [28]. It also adds a bit of reasoning about impact and next steps (check transport network), which IRBE rules alone would not provide.

Ambiguous Scenario Explanation: In one example of an ambiguous case (with both overload symptoms and backhaul problems), the IRBE did not have a specific rule, so the SSC used the term overload. The LLM-generated explanation, on the other hand, added useful detail. It said that the cell was under a lot of stress and that packets were being dropped because the queues were full, which is what happens when there is congestion. It also pointed out "transport timeout" messages that could mean the backhaul was unstable at the same time. Even though the final class was still overloaded, the explanation brought up the secondary backhaul issue, which gave operators a better idea of what was going on. This showcases the LLM's ability to mention observations without being constrained by a single-label output. In doing so, it provided a richer description that a human could interpret as "there might be multiple issues." This is an advantage of using an LLM; it can reflect uncertainty or multiple possibilities naturally [29].

Novel Fault Case: We had a real novel fault example in the data: a Memory Leak issue on a base station (not one of our 5 classes). The logs showed a gradual increase in memory usage and then a process crash. IRBE/SSC did not label it (it was essentially "unseen"). The LAE, given just the logs, returned "*The logs indicate the process memory usage kept growing (memory allocation failures appear) until the process crashed with a Segmentation Fault. This suggests a Memory Leak or software bug causing the node to run out of memory and crash.*" This was a correct analysis purely from the log content. Hy-LIFT would report this as "Novel Fault: possible Memory Leak (software crash)" with that explanation. In our evaluation, this matched the ground truth (the operator had later identified it as a Memory Leak issue). While one anecdote, it demonstrates the system's capability to handle unforeseen problems via the LLM's broad knowledge [30].

User Feedback: We presented 10 explanation outputs (5 IRBE rule explanations, 5 LLM explanations, covering various faults) to three network engineers (not involved in this research) and asked for feedback on clarity and usefulness. The LLM explanations were consistently rated as more helpful. On a 5-point scale (5 = very clear/helpful), LLM explanations averaged 4.6, whereas the terse rule-based ones averaged 3.2. Comments from the engineers included "*The AI-written explanation reads like how I would explain it to a colleague, it saves me time interpreting the raw alarm.*" and "The rule alert is correct but cryptic (just says what triggered). The extended explanation adds context and possible causes, which is great." This aligns with our expectations that natural language, rich explanations improve operator experience [31].

One concern we checked for is the correctness of explanations. In all tested cases, the LLM did not hallucinate any major facts; it mostly stayed factual to the input (likely because the prompt included specific log evidence). Occasionally, it would mention a possible cause that was a bit speculative. For example, in one overload incident, it said "possible cause could be a sudden spike in users or a misconfigured parameter." The actual cause was heavy traffic load, so that is fine, but the mention of misconfiguration was not based on the log. It was not wrong per se, just an additional guess. We consider this acceptable, even potentially useful as a suggestion. However, we advise that such suggestions be taken as hypotheses. Operators generally appreciated the suggestions because they hinted at what to investigate. Nevertheless, to keep trust, one could configure LAE to be more cautious (e.g., phrase such lines as "possibly…" which it did). Ensuring the LLM does not confidently assert unverified information is important for credibility.

Rule vs. LLM Complementarity: It is important to note that the rule-based explanations are deterministic and specific; they point exactly to the condition triggered. The LLM then elaborates on those conditions in a more general narrative. We found this com-

plementary. For audit purposes, one might still log the IRBE rule trigger for traceability while presenting the LLM narrative for action. Hy-LIFT outputs both: e.g., internal logs contain "Fault X detected by Rule Y" and the user-facing output contains the LLM text referencing the same event. This way, if needed, one can always fall back to the concise rule explanation (which is akin to a proof of why the system thought so). Having both ensures scientific rigor (no completely free-form AI decision without backing).

Confusion Matrix and Explanation: Referencing Figure 2 again, the classes with more confusion (coverage vs. overload) are also the ones where explanations can help. If an event was on the fence, say classified as an overload but with some coverage elements, the LLM might mention radio link issues. In one instance, Hy-LIFT misclassified a coverage drop as an overload. The LLM explanation said, "Cell is overwhelmed by load, many RRC rejections due to limited signal reach." This explanation, which was a little confusing, suggested that there was a coverage problem ("limited signal reach"), even though it called it a load. This case shows that the LLM sometimes picks up on clues from the other class. The classification was wrong, but the explanation helped. An engineer reading that might think that it could be a coverage hole instead of just a load. This is a subtle but interesting thing: the LLM explanation can bring out things that the one-hot label hides by talking about evidence. This could lessen the effects of some wrong classifications by giving the raw evidence.

In short, the qualitative evaluation shows that Hy-LIFT's explanations are a big plus. Rule-based logic makes it clear how a fault was found, and the LLM augmentation makes it clear why it matters and what might have caused it. This closes the gap between what machines say and what people understand [32]. The combination addresses the interpretability and actionability requirements that are critical in network operations [1].

The next section will discuss these results in a broader context, including limitations and practical deployment considerations like handling concept drift, runtime performance, and maintaining the system in the long term.

5. Discussion

The assessment of Hy-LIFT shows that it is good at finding network problems and giving clear explanations that people can understand. In this part, we talk about the results and some important parts of the framework, such as how easy it is to understand, how strong it is, and how it can be used in real life (concept drift, scalability), as well as how it compares to other similar approaches. We also talk about the problems with our current implementation and how they could be fixed in the future.

Interpretability and Trust: One of the primary motivations for Hy-LIFT was to ensure that AI-driven fault diagnosis in networks remains interpretable and trustworthy. Our hybrid approach achieves this by design; the IRBE component encodes expert knowledge transparently, and the LAE articulates the reasoning behind each diagnosis in natural language. This stands in contrast to a purely black-box deep learning model that might output an alarm with no justification. The importance of such interpretability is echoed in industry demands for Explainable AI in 5G/6G operations [33]. By combining symbolically explainable elements (rules) with sub-symbolic learning, Hy-LIFT offers what we can term "Glass Box AI" for fault management: the decision process can be inspected at multiple levels (which rule, which features, what explanation). Our user feedback indicated that engineers felt more confident in the system's outputs when accompanied by the LLM explanation, as it resonated with their own thought process. We believe this will ease the adoption of AI in network operations, as it addresses the often-cited issue of ML models being unaccountable or untrusted in critical environments [34].

It is worth noting that while we heavily rely on the LLM for explanation, the LLM's output itself is not guaranteed to be 100% correct (though we saw it was accurate in our cases). This raises an interesting question: if the LLM explanation and the classification disagree, which should be trusted? In our design, the classification label is determined by IRBE/SSC, and the LLM is only there to explain. If it ever said something contradictory (which we did not observe in our test), we would likely treat that as a sign of either a misclassification or a hallucinated explanation. One way to enforce consistency is to include the model's decision in the LLM prompt (which we did). The LLM then usually aligns its narrative with that decision [35]. In practice, we found the LLM often adds nuance but not outright contradiction. For additional safety, one could programmatically check the explanation for mention of a different fault type and flag such cases for review. This could be part of an explanation validation step. The synergy of rules + LLM also provides a form of cross-validation: if an explanation does not match any rule or known pattern, it might indicate the event is novel or the system is unsure. That can help route those cases to human experts, maintaining a human-in-the-loop for outliers.

Robustness to Noise and Ambiguity: Our results indicate that Hy-LIFT is robust to noise in logs and moderate ambiguities. The rule-based part is deterministic (either pattern matches or not; noise is mostly ignored unless it interferes with the pattern). The ML part, being trained on real noisy logs, inherently learns to focus on salient features and ignore irrelevant ones (if those features do not correlate with labels). The LLM, with its pre-training on diverse text, is also robust to extraneous log lines; it can filter out irrelevant lines when forming an explanation (given its large context understanding) [15]. For example, if there is a random debug line in the log, the LLM usually does not mention it unless it looks like part of the issue. This is an advantage of using a powerful language model; it has some built-in understanding of what constitutes an "error" vs. normal messages, as seen in prior log analysis [36].

Regarding class imbalance, Hy-LIFT's performance on minority classes (like hardware failure in our test) remained strong. This is attributable to our conscious steps: IRBE contributed a few high-quality examples of those, SSC's pseudo-labeling multiplied them, and we also balanced the training batches. Without these, a purely data-driven approach might have underperformed on those classes. This demonstrates that incorporating domain knowledge (even if just a handful of examples) can greatly alleviate imbalance issues [37]. A small number of expert-labeled instances of each fault went a long way in our semi-supervised training. This is encouraging for realistic deployments were obtaining even 5–10 examples of each fault type may be feasible through simulation or past incidents.

Concept Drift and Evolving Networks: A practical concern is how Hy-LIFT will fare as the network evolves (software updates, new equipment, new fault types over time). Concept drift can manifest in logs as changes in message format or in the statistical patterns of faults. For instance, a 6G network slice might produce different KPIs than a 5G network did, or an update might change the wording of an error from "Link lost" to "Connection timeout". Our system would need maintenance in such cases:

- **IRBE maintenance:** Rules may need updates if log formats change. This is a known upkeep cost of rule-based systems. However, since IRBE rules are relatively straightforward string matches or simple conditions, updating them is not too onerous if the changes are known (e.g., search-and-replace the keyword). What is more challenging is if entirely new fault types emerge (like a brand-new type of failure). Then, new rules would need to be written, which requires expert identification. In the interim, the ML component or LLM might catch it as an anomaly or novel (zero-shot), as we saw, but for continuous monitoring, eventually adding a rule (or at least adding it as a known class for SSC) would be prudent. Our framework can ingest new rules on the

fly, and the SSC can be periodically retrained with new labels, enabling an incremental learning setup.

- **SSC retraining:** The SSC model can be retrained on a schedule (e.g., weekly or when a drift is detected). It can incorporate newly accumulated data, including any new fault labels (from operator feedback or LAE suggestions). Online/sequential learning techniques could be used for faster adaptation. Additionally, anomaly detection on model inputs could flag drift if the distribution of features shifts significantly, which might trigger retraining. In our design, since IRBE is always looking for known patterns, it acts as a stable reference. If the ML starts flagging many things that IRBE would have, perhaps the rules need expansion. Conversely, if IRBE stops triggering because logs have changed superficially, one will notice a drop and can adjust the rules.
- **LLM adaptation:** The LLM we used (GPT-4-Class) is a static model with knowledge up to 2021. In a future scenario, if new fault terminology arises (say, 6G-specific jargon), the LLM might not be aware. However, one can fine-tune or prompt-engineer it with updated knowledge. For example, providing it with a glossary of new error codes or feeding in some examples (few-shot) of new fault explanations could keep it up to date. Another approach is using a retrieval-augmented generation (RAG), where the LLM has access to a knowledge base (like documentation or a log knowledge graph) [15]. Then, for any unknown term, it could query that. In our current implementation, we did not use retrieval, but it is a promising extension for reliability (ensuring the LLM does not hallucinate and uses up-to-date info). Incorporating an internal knowledge base of network faults that the LLM can reference would make explanations even more precise. In fact, Isaac et al. (2025) used an LLM trained on 5G core documents to suggest fixes, demonstrating that domain-specific tuning yields actionable advice [38]. We could similarly fine-tune our LAE on, say, a corpus of historical incident reports to sharpen its explanations.

In short, Hy-LIFT can handle concept drift and evolution by regularly updating rules and models. The fact that it has separate IRBE, SSC, and LAE modules means that each part can be updated on its own when needed, which is a plus. With a monolithic model, you would have to retrain the whole thing, or it might slowly get worse. Our method lets you make specific improvements (for example, you can add a new rule for a new fault and immediately cover it with an interpretation).

Scalability and Performance: Another practical matter is how well this approach scales to large networks and real-time processing. Let us consider each component:

- IRBE is very fast and lightweight; it is essentially a set of pattern matches. We implemented it streaming through logs with negligible overhead (on the order of milliseconds per log entry). This can easily scale to thousands of logs per second, which is typical in telecom.
- SSC classification, once trained, is also quick. A Random Forest or small neural net can classify in microseconds per sample. The training of SSC (with pseudo-label iterations) is more expensive, but that can be carried out offline and perhaps on batch data. We did pseudo-labeling on approximately 5000 unlabeled samples in seconds; even for millions, it would be manageable with sufficient compute and maybe sampling.
- LAE is the most computationally heavy and potentially a bottleneck if used on every event. GPT-4-Class via API took ~2 s per explanation. If the system were to generate explanations for hundreds of alarms per minute, doing that sequentially would be an issue. However, in practice, critical fault alarms in a network are not that frequent, maybe a few per hour in normal conditions. It is feasible to handle those with an API call. If one needed to generate explanations for a flood of minor alerts, one could use a

smaller LLM (e.g., GPT-3 or an open model), which can be faster or run multiple in parallel. We can also prioritize, perhaps, only use the LLM for significant incidents, whereas minor ones can just use rule text or a template. Since our focus is on major fault diagnosis, the latency of a couple of seconds is acceptable in an operational sense (it is still far quicker than human analysis, which might take minutes or hours). In any case, with model optimization and caching (common patterns explained once), the LAE overhead can be mitigated. Running a local instance of a model (like Llama-2) on a GPU could cut the explanation time to sub-second for shorter outputs based on some experiments we carried out with the smaller model (not reported in detail).

Comparison with Other Approaches: It is useful to position Hy-LIFT relative to the alternatives:

- End-to-End Deep Learning: Approaches where a deep model directly predicts faults from logs (e.g., CNN/RNN on raw log sequences). Such models (like those referenced in surveys [16]) can achieve high accuracy if well-trained, as in some BERT-based log classifiers hitting F1 > 0.95 on benchmarks. However, they typically need large, labeled datasets and are black boxes in nature. Our results (F1 ~0.89) may be slightly lower than the best deep models on simpler tasks, but we trade a bit of accuracy for explainability and low label requirement. In a domain like telecom, that trade-off is often worth it [1]. Moreover, our approach could incorporate such deep models in the SSC stage if the data allows the framework to be agnostic to the classifier type. We used RF for ease and interpretability, but one could plug in a Transformer-based log anomaly detector and keep the IRBE and LAE around it for explainability. In fact, using something like LogBERT [15] as the SSC could potentially raise the accuracy closer to 95% on known classes, and IRBE/LAE would guard and explain it [39]. This hybrid-within-hybrid is an interesting future path: use state-of-the-art deep anomaly detectors for raw power and our rule/LLM to temper and elucidate them.
- Pure Expert Systems vs. Pure LLM Solutions: On one end, older expert systems (rules only) are highly trusted but inflexible [2]. On the other hand, a stand-alone version of "ChatGPT for logs" might give you plausible analyses, but you cannot be sure that they will be consistent or complete. Hy-LIFT finds a balance, rules keep the system in check, and LLM adds flexibility. We do not rely too much on the LLM because we do not use it to make the main fault decision directly (that is, SSC's job, except for new cases). This was carried out on purpose because current LLMs are smart, but they are not specialized or consistent enough to take the place of a trained classifier for known categories. Egersdoerfer et al. observed that ChatGPT occasionally misclassified without guidance [13]. Our approach ensures the LLM works in tandem with a reliable pattern detector and a learned model. This follows recommendations from studies like the log analysis survey [15] which suggest combining parsing, ML, and LLM for best results, rather than using an LLM blindly.
- Similar Frameworks: Tang et al.'s LLM-assisted and IDS-Agent [17] are conceptually similar in layering an analysis model with an explanation model [15]. Tang achieved a high detection accuracy (91%) focusing on heterogeneous network anomalies; IDS-Agent demonstrated improved zero-day detection using GPT. Hy-LIFT's novelty is particularly in the semi-supervised learning integration and explicit rule-based stage. Neither of those works utilized a rule engine; Tang's used attention-based semantic rules inside a model (less interpretable to humans, more like features), and IDS-Agent used multiple ML models, but still a black-box ensemble. We believe Hy-LIFT's explicit, interpretable first stage is a distinguishing factor for real-world deployment, where existing rule systems are already in place (we effectively can augment an existing alarm system with learning and LLM layers, rather than replacing

it). The strong performance of Hy-LIFT suggests that even if one has a sophisticated anomaly detector, adding a rule knowledge base can enhance it, an idea supported by Zhao et al.'s fusion results [4].

- Scalability and Replicability: Even though Hy-LIFT integrates several modules, it remains straightforward to deploy and operate in practice. The IRBE performs deterministic rule matching in only a few milliseconds per log entry, and the SSC (implemented as a Random Forest classifier) generates predictions in the microsecond range, so both components scale comfortably to high-volume log streams. The LLM-based explanation engine (LAE) is comparatively more expensive computationally, but it is only triggered for ambiguous or high-impact cases and therefore adds limited overhead on the order of roughly two seconds per explanatory query when using the GPT-4 API. We also outline strategies for scaling the LAE when needed, such as using more efficient language models or parallelizing inference during alarm bursts. Importantly, Hy-LIFT is data-efficient: strong performance was achieved with only ~1.2 k labeled logs by leveraging thousands of unlabeled logs through semi-supervised learning. This reduces the operational burden of acquiring large, annotated datasets. Moreover, the modular architecture enables straightforward transfer to new networks or domains; only the rule set and classifier need retraining, while the pipeline itself remains unchanged. These additions clarify that the framework is not only accurate and interpretable but also scalable, replicable, and viable for deployment in real 5G/6G environments [40,41].
- We have added a dedicated discussion on how Hy-LIFT can be applied to other operational contexts such as cloud systems, enterprise IT infrastructures, or IoT networks. The modular architecture of Hy-LIFT inherently supports domain transfer. Specifically, one can update the IRBE rule set to capture domain-specific signatures (e.g., rules for VM provisioning failures or IoT sensor communication faults), retrain the SSC on logs from the new environment using the same semi-supervised approach to handle scarce labeled data, and optionally fine-tune the LLM-based explanation component with domain terminology or knowledge. Importantly, no architectural modifications are required; each module (IRBE, SSC, LAE) can be adapted independently. This modular reconfiguration is significantly easier and more maintainable than redesigning an end-to-end monolithic model. By plugging in domain-relevant rules and data, Hy-LIFT can seamlessly migrate to cloud, enterprise, or IoT scenarios, demonstrating strong extensibility and practical value beyond 5G/6G network settings [42].

Limitations: While Hy-LIFT performed well in our study, we acknowledge some limitations:

- It currently handles only the classification of faults of types it knows, plus a generic handling of unknowns. It does not carry out root cause analysis (RCA) beyond identifying the fault category and immediate cause. True RCA might require correlating multiple events across network elements (e.g., pinpointing that a router failure caused many cell outages). Our framework could potentially assist in RCA by using LLM to connect the dots between multiple fault instances (LLMs are good at reading and summarizing multiple inputs), but we have not explicitly built cross-correlation logic. A next step could be feeding the LLM with combined logs from several cells to say "these 5 cells went down around the same time, possibly due to a common backhaul node failure," essentially letting it hypothesize higher-level causes. There is ongoing research in telecom RCA using AI (e.g., Bayesian networks, graph analysis); integrating that with our approach could be fruitful.
- The quality of IRBE rules and initial labels heavily influences performance. If we missed an important pattern in rules, and it is also rare in data, SSC might never

learn it. The framework is not magically going to detect something where there is zero signal. That said, the LLM might flag something as strange if truly anomalous, even if no class, but it may not label it correctly. Thus, Hy-LIFT is only as good as the knowledge it is given, plus what it can infer from the data. We tried to simulate a realistic scenario, but in a new domain or a very novel fault, human input would still be needed to incorporate that knowledge.

- Another limitation is the dependency on an external LLM (if using a closed API like GPT-4-Class). We were worried about data privacy (we protected it by anonymizing it and could choose on-prem models) and consistency (the LLM might act differently with updates or different prompts). We saw the LLM as a module that could be replaced; you could use an in-house model instead. One thing we did was use prompt engineering to make sure that the output was always in the same format (for example, we told it to start with a summary sentence, etc.). In production, you might want even more structured explanations, like a set template or JSON with fields for cause, effect, and recommendation. Right now, ours are free-form paragraphs. You could understand all of them, but NOC workflows need things to be clear and short (you do not want a page-long summary). We could look into prompt constraints or fine-tuning to make the outputs more consistent.

Ethical and Deployment Considerations: When using an AI system in telecom operations, you need to think about how reliable it is and how to make sure it does not fail. To compare its alerts to those of older systems, Hy-LIFT can be run alongside them at first. It could speed up the validation process for engineers because it gives explanations. We want to make it clear that the system is not fully automatic control; it only helps people make decisions. However, it could start automated fixes for known problems, but those fixes would probably be based on rules, like rebooting a cell when there is a hardware problem. The explanations are especially useful for training new employees and keeping knowledge alive, as they record the reasons behind an event as it happens. This fixes a problem with operations where you need a lot of tacit knowledge to understand alarms.

Finally, we compare our approach to the broader trend of hybrid AI. In various domains, combining symbolic AI (rules/knowledge graphs) with sub-symbolic AI (neural networks) and now large pre-trained models is seen to get the best of both worlds [43]. Our work is a concrete instantiation of that philosophy in the context of network management. The success observed here encourages further exploration of hybrid designs for other complex system diagnostics (e.g., cloud systems, IoT networks) [44]. We anticipate that, moving into 6G and beyond, where networks are highly autonomous (zero-touch management), such hybrid explainable systems will be crucial for operators to maintain oversight and trust in automation.

6. Conclusions

We presented Hy-LIFT, a Hybrid LLM-Assisted Fault Diagnosis Framework for 5G/6G networks, which integrates an interpretable rule-based engine, a semi-supervised classifier, and an LLM-based explanation module. Hy-LIFT was designed to meet the twin challenges of high accuracy and interpretability in analyzing real-world network logs. Through a comprehensive evaluation on operational 5G log data, we demonstrated that Hy-LIFT can effectively detect a range of network faults (coverage issues, handover failures, backhaul outages, congestion, hardware faults) with high overall accuracy (~89–90% macro-F1), significantly outperforming standalone rule-based or machine learning approaches [45]. The framework proved robust to noisy log inputs and class imbalance, owing to its fusion of expert knowledge (precision-focused rules) and data-driven learning (recall-boosting semi-supervision).

A distinctive feature of our system is its ability to generate human-understandable explanations for each detected fault. By leveraging a large language model in the LAE stage, Hy-LIFT produces narrative diagnostic reports that contextualize faults and often suggest likely causes or remedies [14]. This was validated by network engineers to greatly enhance the usability of the system's outputs, bridging the gap between raw alarms and actionable insight. Additionally, the LLM augmentation allowed the framework to handle novel faults in a zero-shot manner, providing at least a tentative diagnosis for previously unseen issues, which is crucial in evolving networks.

Hy-LIFT is a good example of a hybrid AI solution that meets the needs of modern telecom operations. It keeps the openness of traditional rule-based O&M systems while adding the flexibility and inference power of machine learning and the depth of explanation of LLMs. You can use this kind of system alongside other network management tools, and it will gradually take on more responsibility as people trust its diagnoses. We think this method will cut down on the time it takes to diagnose problems by automatically analyzing logs and pointing operators to the root causes faster than doing it by hand. This will make the network more reliable and cut down on downtime.

This work also contributes to the broader field of explainable AI for IT operations (AIOps). We showed that combining expert knowledge with semi-supervised learning is a viable strategy to overcome the paucity of labeled data in many industrial settings, not just telecom [46,47]. Moreover, we demonstrated a concrete method to incorporate large language models in a controlled, beneficial way using them as explainers and reasoning engines on top of structured detections, rather than naively replacing the detection logic. This approach yielded explanations that were both accurate and comprehensible, avoiding the pitfalls of uninterpretable models or unfettered language models.

Hy-LIFT will grow in three main ways in the future. We will first add a root cause analysis layer that links multiple related alarms to find higher-level causes. For example, we will group cell failures to find a shared backhaul problem and use the LLM to reason over multi-entity logs. Second, we want to make the learning part stronger by looking into graph-based and active semi-supervised methods, as well as online updates to deal with concept drift as new fault patterns show up. Third, we will look into domain-specific and on-premises LLMs, as well as a user study in a live NOC setting, to see how much the explanations speed up resolution time and make operators feel more confident. Because Hy-LIFT is modular, we also plan to change the rules and retrain the classifier on logs that are specific to the domain so that the framework can be used for other tasks, like monitoring performance and finding security anomalies.

Hy-LIFT shows that using interpretable rules, semi-supervised learning, and LLM-based explanations together is a good way to find faults in 5G and 6G networks. The framework works well even when there are a lot of different classes, and it makes sure that operators can understand and check the decisions. Its modular design makes it easy to add to existing O&M workflows and make it work with other networked environments. As networks become more independent and complicated, these kinds of hybrid, understandable toolkits can help keep control that is based on data without losing human oversight.

Author Contributions: Conceptualization, A.T.Z. and A.D.S.; methodology, A.T.Z. and F.H.J.; software, A.T.Z.; validation, A.T.Z. and S.S.J.; formal analysis, A.J.H. and S.M.R.; investigation, A.D.S. and F.H.J.; resources, A.T.Z.; data curation, A.T.Z.; writing—original draft preparation, A.T.Z.; writing—review and editing, A.D.S. and S.S.J.; visualization, A.T.Z. and S.M.R.; supervision, A.J.H., S.M.R. and A.D.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The network log datasets used in this study contain sensitive operational data and cannot be made publicly available. However, we have provided a representative synthetic log dataset and the Hy-LIFT code on GitHub (https://github.com/Akramtaha98/Hy-LIFT-LLM-Fault-Diagnosis.git (accessed on 10 November 2025)). This allows interested researchers to reproduce the methodology and test the framework on their own data.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

5GC	5G Core
6G	Sixth-Generation Mobile Networks
gNB	Next-Generation NodeB (5G base station)
UE	User Equipment
UPF	User Plane Function (5GC)
RRC	Radio Resource Control
SINR	Signal-to-Interference-plus-Noise Ratio
KPI	Key Performance Indicator
QoS	Quality of Service
Hy-LIFT	Hybrid LLM-Integrated Fault Diagnosis Toolkit
IRBE	Interpretable Rule-Based Engine
SSC	Semi-Supervised Classifier
LAE	LLM Augmentation Engine
LLM	Large Language Model
XAI	Explainable Artificial Intelligence

References

1. Senevirathna, T.; La, V.H.; Marcha, S.; Siniarski, B.; Liyanage, M.; Wang, S. A Survey on XAI for 5G and Beyond Security: Technical Aspects, Challenges and Research Directions. *IEEE Commun. Surv. Tutor.* **2025**, *27*, 941–973. [CrossRef]
2. Qian, B.; Lu, S. Detection of Mobile Network Abnormality Using Deep Learning Models on Massive Network Measurement Data. *Comput. Netw.* **2021**, *201*, 108571. [CrossRef]
3. Amuah, E.A.; Wu, M.; Zhu, X. Cellular Network Fault Diagnosis Method Based on a Graph Convolutional Neural Network. *Sensors* **2023**, *23*, 7042. [CrossRef]
4. Zhao, L.; He, C.; Zhu, X. A Fault Diagnosis Method for 5G Cellular Networks Based on Knowledge and Data Fusion. *Sensors* **2024**, *24*, 401. [CrossRef] [PubMed]
5. Huang, S.; Liu, Y.; Fung, C.; He, R.; Zhao, Y.; Yang, H.; Luan, Z. Transfer Log-Based Anomaly Detection with Pseudo Labels. In Proceedings of the 2020 16th International Conference on Network and Service Management (CNSM), Izmir, Turkey, 2–6 November 2020; pp. 1–5.
6. Yang, L.; Chen, J.; Wang, Z.; Wang, W.; Jiang, J.; Dong, X.; Zhang, W. PLELog: Semi-Supervised Log-Based Anomaly Detection via Probabilistic Label Estimation. In Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), Madrid, Spain, 25–28 May 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 230–231.
7. Sangaiah, A.K.; Rezaei, S.; Javadpour, A.; Miri, F.; Zhang, W.; Wang, D. Automatic Fault Detection and Diagnosis in Cellular Networks and Beyond 5G: Intelligent Network Management. *Algorithms* **2022**, *15*, 432. [CrossRef]
8. Khatib, E.J.; Barco, R.; Gomez-Andrades, A.; Serrano, I. Diagnosis Based on Genetic Fuzzy Algorithms for LTE Self-Healing. *IEEE Trans. Veh. Technol.* **2016**, *65*, 1639–1651. [CrossRef]
9. Szilagyi, P.; Novaczki, S. An Automatic Detection and Diagnosis Framework for Mobile Communication Systems. *IEEE Trans. Netw. Serv. Manag.* **2012**, *9*, 184–197. [CrossRef]
10. Uszko, K.; Kasprzyk, M.; Natkaniec, M.; Chołda, P. Rule-Based System with Machine Learning Support for Detecting Anomalies in 5G WLANs. *Electronics* **2023**, *12*, 2355. [CrossRef]
11. Wang, X.; Fu, Z.; Li, X. A Graph Deep Learning-Based Fault Detection and Positioning Method for Internet Communication Networks. *IEEE Access* **2023**, *11*, 102261–102270. [CrossRef]
12. Ahmad, A.; Li, P.; Piechocki, R.; Inacio, R. Anomaly Detection in Offshore Open Radio Access Network Using Long Short-Term Memory Models on a Novel Artificial Intelligence-Driven Cloud-Native Data Platform. *Eng. Appl. Artif. Intell.* **2025**, *161*, 112274. [CrossRef]

13. Han, X.; Yuan, S.; Trabelsi, M. LogGPT: Log Anomaly Detection via GPT. In Proceedings of the 2023 IEEE International Conference on Big Data (BigData), Sorrento, Italy, 15–18 December 2023.
14. Egersdoerfer, C.; Zhang, D.; Dai, D. Early Exploration of Using ChatGPT for Log-Based Anomaly Detection on Parallel File Systems Logs. In Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing, Orlando, FL, USA, 16–23 June 2023; Association for Computing Machinery: New York, NY, USA, 2023; pp. 315–316.
15. Akhtar, S.; Khan, S.; Parkinson, S. LLM-Based Event Log Analysis Techniques: A Survey. *arXiv* **2025**, arXiv:2502.00677. [CrossRef]
16. Tang, F.; Wang, X.; Yuan, X.; Luo, L.; Zhao, M.; Huang, T.; Kato, N. Large Language Model(LLM) Assisted End-to-End Network Health Management Based on Multi-Scale Semanticization. *arXiv* **2025**, arXiv:2406.08305. [CrossRef]
17. Ali, T.; Kostakos, P. HuntGPT: Integrating Machine Learning-Based Anomaly Detection and Explainable AI with Large Language Models (LLMs). *arXiv* **2023**, arXiv:2309.16021. [CrossRef]
18. Jirjees, S.W.; Alkhalid, F.F.; Hasan, A.M.; Humaidi, A.J. A Secure Password based Authentication with Variable Key Lengths Based on the Image Embedded Method. *Mesopotamian J. Cybersecur.* **2025**, *5*, 491–500. [CrossRef]
19. Li, Y.; Xiang, Z.; Bastian, N.D.; Song, D.; Li, B. IDS-Agent: An LLM Agent for Explainable Intrusion Detection in IoT Networks. *arXiv* **2025**, arXiv:2510.13925. [CrossRef]
20. Senevirathna, T.; Sandeepa, C.; Siniarski, B.; Nguyen, M.-D.; Marchal, S.; Boerger, M.; Liyanage, M.; Wang, S. Enhancing Accountability, Resilience, and Privacy of Intelligent Networks With XAI. *IEEE Open J. Commun. Soc.* **2025**, *6*, 8389–8409. [CrossRef]
21. Karahan, S.N.; Güllü, M.; Karhan, D.; Çimen, S.; Osmanca, M.S.; Barışçı, N. Realistic Performance Assessment of Machine Learning Algorithms for 6G Network Slicing: A Dual-Methodology Approach with Explainable AI Integration. *Electronics* **2025**, *14*, 3841. [CrossRef]
22. Salman, A.D.; Zeyad, A.T.; Al-karkhi, A.A.S.; Raafat, S.M.; Humaidi, A.J. Hybrid CDN Architecture Integrating Edge Caching, MEC Offloading, and Q-Learning-Based Adaptive Routing. *Computers* **2025**, *14*, 433. [CrossRef]
23. Liu, K.; Ling, S.; Liu, S. Semi-Supervised Medical Image Classification with Pseudo Labels Using Coalition Similarity Training. *Mathematics* **2024**, *12*, 1537. [CrossRef]
24. He, Y.; Pei, X. Semi-Supervised Learning via DQN for Log Anomaly Detection. *arXiv* **2024**, arXiv:2401.03151. [CrossRef]
25. Corona, J.; Rodrigues, P.; Almeida, L.; Teixeira, R.; Antunes, M.; Aguiar, R. From Black Box to Transparency: The Hidden Costs of XAI in NGN. In Proceedings of the 2024 IEEE Globecom Workshops (GC Wkshps), Cape Town, South Africa, 8–12 December 2024.
26. Li, J.; Zhu, K.; Zhang, Y. Knowledge-Assisted Few-Shot Fault Diagnosis in Cellular Networks. In Proceedings of the 2022 IEEE Globecom Workshops (GC Wkshps), Janeiro, Brazil, 4–8 December 2022; pp. 1292–1297.
27. Moulay, M.; Leiva, R.G.; Maroni, P.J.R.; Lazaro, J.; Mancuso, V.; Anta, A.F. A Novel Methodology for the Automated Detection and Classification of Networking Anomalies. In Proceedings of the IEEE INFOCOM 2020—IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), Toronto, ON, Canada, 6–9 July 2020; pp. 780–786.
28. Qi, J.; Huang, S.; Luan, Z.; Yang, S.; Fung, C.; Yang, H.; Qian, D.; Shang, J.; Xiao, Z.; Wu, Z. LogGPT: Exploring ChatGPT for Log-Based Anomaly Detection. In Proceedings of the 2023 IEEE International Conference on High Performance Computing & Communications, Data Science & Systems, Smart City & Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys), Melbourne, Australia, 17–21 December 2023; pp. 273–280.
29. Zhang, Z.; Li, S.; Zhang, L.; Ye, J.; Hu, C.; Yan, L. LLM-LADE: Large Language Model-Based Log Anomaly Detection with Explanation. *Knowl. Based Syst.* **2025**, *326*, 114064. [CrossRef]
30. Huang, S.; Liu, Y.; Fung, C.; He, R.; Zhao, Y.; Yang, H.; Luan, Z. Paddy: An Event Log Parsing Approach Using Dynamic Dictionary. In Proceedings of the NOMS 2020—2020 IEEE/IFIP Network Operations and Management Symposium, Budapest, Hungary, 20–24 April 2020; pp. 1–8.
31. Cui, T.; Ma, S.; Chen, Z.; Xiao, T.; Tao, S.; Liu, Y.; Zhang, S.; Lin, D.; Liu, C.; Cai, Y.; et al. LogEval: A Comprehensive Benchmark Suite for Large Language Models In Log Analysis. *arXiv* **2024**, arXiv:2407.01896. [CrossRef]
32. Wang, S.; Qureshi, M.A.; Miralles-Pechuán, L.; Huynh-The, T.; Gadekallu, T.R.; Liyanage, M. Explainable AI for 6G Use Cases: Technical Aspects and Research Challenges. *IEEE Open J. Commun. Soc.* **2024**, *5*, 2490–2540. [CrossRef]
33. Siriwardhana, Y.; Porambage, P.; Liyanage, M.; Ylianttila, M. AI and 6G Security: Opportunities and Challenges. In Proceedings of the 2021 Joint European Conference on Networks and Communications & 6G Summit (EuCNC/6G Summit), Porto, Portugal, 8–11 June 2021; pp. 616–621.
34. Wang, S.; Qureshi, M.A.; Miralles-Pechuán, L.; Huynh-The, T.; Gadekallu, T.R.; Liyanage, M. Applications of Explainable AI for 6G: Technical Aspects, Use Cases, and Research Challenges. *arXiv* **2023**, arXiv:2112.04698. [CrossRef]
35. Shiri, F.; Moghimifar, F.; Haffari, R.; Li, Y.-F.; Nguyen, V.; Yoo, J. Decompose, Enrich, and Extract! Schema-Aware Event Extraction Using LLMs. In Proceedings of the 2024 27th International Conference on Information Fusion (FUSION), Venice, Italy, 8–11 July 2024; pp. 1–8.

36. Guan, W.; Cao, J.; Qian, S.; Gao, J.; Ouyang, C. LogLLM: Log-Based Anomaly Detection Using Large Language Models. *arXiv* **2025**, arXiv:2411.08561. [CrossRef]
37. Porch, J.B.; Heng Foh, C.; Farooq, H.; Imran, A. Machine Learning Approach for Automatic Fault Detection and Diagnosis in Cellular Networks. In Proceedings of the 2020 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom), Odessa, Ukraine, 26–29 May 2020; pp. 1–5. [CrossRef]
38. Isaac, J.H.R.; Saradagam, H.; Pardhasaradhi, N. 5G Core Fault Detection and Root Cause Analysis Using Machine Learning and Generative AI. *arXiv* **2025**, arXiv:2508.09152. [CrossRef]
39. Zhong, A.; Mo, D.; Liu, G.; Liu, J.; Lu, Q.; Zhou, Q.; Wu, J.; Li, Q.; Wen, Q. LogParser-LLM: Advancing Efficient Log Parsing with Large Language Models. In Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Barcelona, Spain, 25–29 August 2024; Association for Computing Machinery: New York, NY, USA, 2024; pp. 4559–4570.
40. Abbas, O.K.; Abdullah, F.; Radzi, N.A.M.; Salman, A.D.; Abdulkadir, S.J. Survey on Clustered Routing Protocols Adaptivity for Fire Incidents: Architecture Challenges, Data Losing, and Recommended Solutions. *IEEE Access* **2024**, *12*, 113518–113552. [CrossRef]
41. Ola, O.; Abdullah, F.; Radzi, N.A.M.; Salman, A.D. New Adaptive-Clustered Routing Protocol for Indoor Fire Emergencies Using Hybrid CNN-BiLSTM Model: Development and Validation. *J. Intell. Syst. Internet Things* **2025**, *14*, 08–24. [CrossRef]
42. Al-Ani, A.; Seitz, J. An Approach for QoS-Aware Routing in Mobile Ad Hoc Networks. In Proceedings of the 2015 International Symposium on Wireless Communication Systems (ISWCS), Brussels, Belgium, 25–28 August 2015; IEEE: Piscataway, NJ, USA, 2015; pp. 626–630.
43. Zhu, X.; Zhao, L.; Cao, J.; Cai, J. Fault Diagnosis of 5G Networks Based on Digital Twin Model. *China Commun.* **2023**, *20*, 175–191. [CrossRef]
44. Jasim, Z.M.; Salman, A.D. Cloud-Based Voice Home Automation System Based on Internet of Things. *Iraqi J. Sci.* **2022**, *63*, 843–854. [CrossRef]
45. Reddy, S.P.V.V.; Juliet, A.H.; Jayadurga, R.; Sethu, S. A Novel Method to Identify and Recover the Fault Nodes over 5G Wireless Sensor Network Environment. In Proceedings of the 2024 Asia Pacific Conference on Innovation in Technology (APCIT), Mysore, India, 26–27 July 2024; pp. 1–6.
46. Wan, Z.; Lin, L.; Huang, Y.; Wang, X. A Graph Neural Network Based Fault Diagnosis Strategy for Power Communication Networks. *J. Chin. Inst. Eng.* **2024**, *47*, 273–282. [CrossRef]
47. Abed, R.A.; Hamza, E.K.; Humaidi, A.J. A Modified CNN-IDS Model for Enhancing the Efficacy of Intrusion Detection System. *Meas. Sens.* **2024**, *35*, 101299. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.